Sécurisation des échanges dans un cœur 5G SA

Encadrant : **C. Destré**

Etudiants : **A. Moniot, M. Mounpain Ndam, Y. N'Gbe-Baï, A. Nousfi**

Table des matières

1 Cahier des charges 2
2 Plan de développement 3
2.1 Etude des documentations techniques 3
2.2 Mise en place de l'environnement expérimental Free5GC et tests 3
2.3 Etude comparative 3
2.4 Recommandations et conclusion 4
2.5 Tâches du projet – Diagramme de Gantt 4
3 Etat de l'art 5
3.1 Introduction à 5G SA 5
3.2 Authentification mutuelle des fonctions 7
3.3 Le protocole TLS 7
3.4 Les exigences de 3GPP pour l'utilisation de TLS dans le cœur 5G SA 9
4 Analyse 12
4.1 Expression des besoins 12
4.2 Descriptif détaillé des tâches 12
4.3 Description des problèmes à résoudre 17
5 Conception 18
5.1 Description de l'environnement expérimental 18
5.2 Logiciels et outils utilisés 20
5.3 Configuration de l'environnement 23
6 Compte rendu du projet 27
6.1 Déroulement du projet 27
6.2 Résultats obtenus 27
6.3 Conclusion 36
7 Références 38
8 Exemple de fichier docker-compose 42
9 Annexe B – Sécurité des suites cryptographiques 43
10 Annexe C – Capture des échanges entre AMF et AUSF non déchiffrés 44

1 Cahier des charges

La 5G est la 5ème génération de réseau mobile. 5G SA pour « Stand Alone » utilise le réseau d'accès radio (RAN) 5G avec un réseau de cœur 5G. Ce réseau de cœur se caractérise par une architecture « Service Based Architecture » comportant plusieurs fonctions comme AUSF (Authentication server function), SMF (session management function) ou PCF (policy control function). La sécurisation des échanges entre ces fonctions est un enjeu crucial pour assurer la confidentialité, l'intégrité et l'authenticité des communications. Les normes 3GPP dans ses releases 16 et 17 imposent l'usage du protocole HTTP/2 pour ces échanges et l'utilisation de TLS renforce la protection des communications mais complexifie l'inspection et l'analyse du trafic, rendant nécessaires des outils et méthodologies adaptés.

Dans ce travail, nous nous concentrerons sur l'analyse et la mise en œuvre des mécanismes de sécurisation des communications au sein d'un cœur 5G SA. Il s'agira de comprendre le fonctionnement du protocole TLS, d'identifier ses vulnérabilités et d'analyser les exigences des spécifications 3GPP en matière de chiffrement et de protection des flux dans un cœur 5G SA. Ainsi, nous déploierons un environnement expérimental basé sur Free5GC et Open5GS afin d'évaluer leur conformité aux normes et de proposer des recommandations pour renforcer la sécurité. Aussi, nous analyserons l'impact de l'activation du chiffrement TLS sur les performances du réseau.

Dans un premier temps nous explorerons les spécifications techniques exprimées dans les releases 16 et 17 de la norme 3GPP relatives à l'emploi du protocole TLS pour la sécurisation des échanges entre les fonctions du plan de contrôle et les exigences par rapport aux primitives de chiffrement et d'échange de clés.

Ensuite, l'analyse des documentations de Free5GC et Open5GS permettra de déterminer leur niveau de conformité quant aux standards en vigueur.

Enfin, les technologies Free5GC et Open5GS seront déployés dans des environnements conteneurisés. Le simulateur de terminaux et de réseau radio UERANSIM sera également déployé permettant de générer du trafic dans le cœur 5G. Des outils spécifiques permettront d'inspecter les échanges TLS entre les différentes fonctions de réseau durant les étapes des différentes procédures telles que l'attachement d'un mobile ou l'enregistrement dans le réseau et d'en évaluer le respect des exigences des spécification de 3GPP.

La conclusion de ce travail sera d'une part une étude comparative mettant en lumière les écarts éventuels entre ces deux implémentations et les spécifications de 3GPP et d'autre part la formulation de recommandations d'amélioration en matière de sécurisation et/ou d'implémentation.

2 Plan de développement

2.1 Etude des documentations techniques

Durant cette phase du projet, nous allons étudier les standards relatifs au déploiement du réseau cœur 5G SA, selon les versions 16 et 17 de la norme 3GPP et plus particulièrement les exigences concernant les protocoles HTTP/2 et TLS utilisés pour les échanges entre les fonctions de réseau virtualisées qui le compose. Cette étude permettra d'établir un référentiel quant aux configurations TLS demandées par 3GPP.

Le protocole TLS est très répandu, mais comporte des vulnérabilités. L'étude du fonctionnement de TLS et des principales vulnérabilités qui le cible sera réalisée permettant d'établir une liste de vulnérabilités qui servira à déterminer dans quelle mesure les implémentations de cœur 5G SA que nous testerons lors de la phase expérimentale sont exposé à des attaques via ces failles.

Enfin, dans une optique de déploiement d'infrastructures de tests, nous détaillerons les documentations de deux implémentations de cœurs 5G SA que sont Free5GC et Open5GS. Cette étude documentaire permettra également de déterminer s'ils respectent les exigences du 3GPP.

2.2 Mise en place de l'environnement expérimental Free5GC et tests

Cette phase concerne la mise en place des infrastructures expérimentales et les tests de la sécurité du plan de contrôle. Le déploiement de l'outil Free5GC sera réalisé dans un environnement conteneurisé Docker et permettra de simuler un cœur de réseau 5G SA. Les fonctions réseau AMF, AUSF, SMF, UDM, UDR, NRF, NSSF, PCF y seront activées. L'outil UERANSIM sera déployé dans une seconde machine virtuelle afin de simuler un réseau accès radio 5G ainsi qu'un terminal 5G et permettra de générer du trafic dans le cœur 5G.

Dans un premier temps, des outils tels que TLS-Scanner et SSLyze permettront d'analyser les certificats et les configurations TLS des différentes fonctions du plan de contrôle comme notamment la version de TLS et les suites cryptographiques proposés. Ils permettront également d'identifier les éventuels vulnérabilités les affectants.

Dans un second temps, nous observerons les échanges entre les différentes NF en capturant les paquets grâce à la sonde Wireshark durant différentes procédures. Ces captures permettront également de récupérer des métriques de performances.

2.3 Etude comparative

Le même protocole de tests sera utilisé sur l'implémentation de réseau de cœur 5G SA Open5GS. Le simulateur RAN et UE ainsi que les outils de tests utilisés seront les mêmes que pour Free5GC.

Nous mènerons une première phase d'analyse comparative entre Free5GC, Open5GS et les spécifications 3GPP qui permettra de mettre en lumière les écarts éventuels relatifs à l'utilisation et la configuration de TLS entre les deux implémentations de réseau de cœur 5G testées et les normes 3GPP.

Une seconde phase de l'analyse permettra de comparer l'impact de l'utilisation de TLS

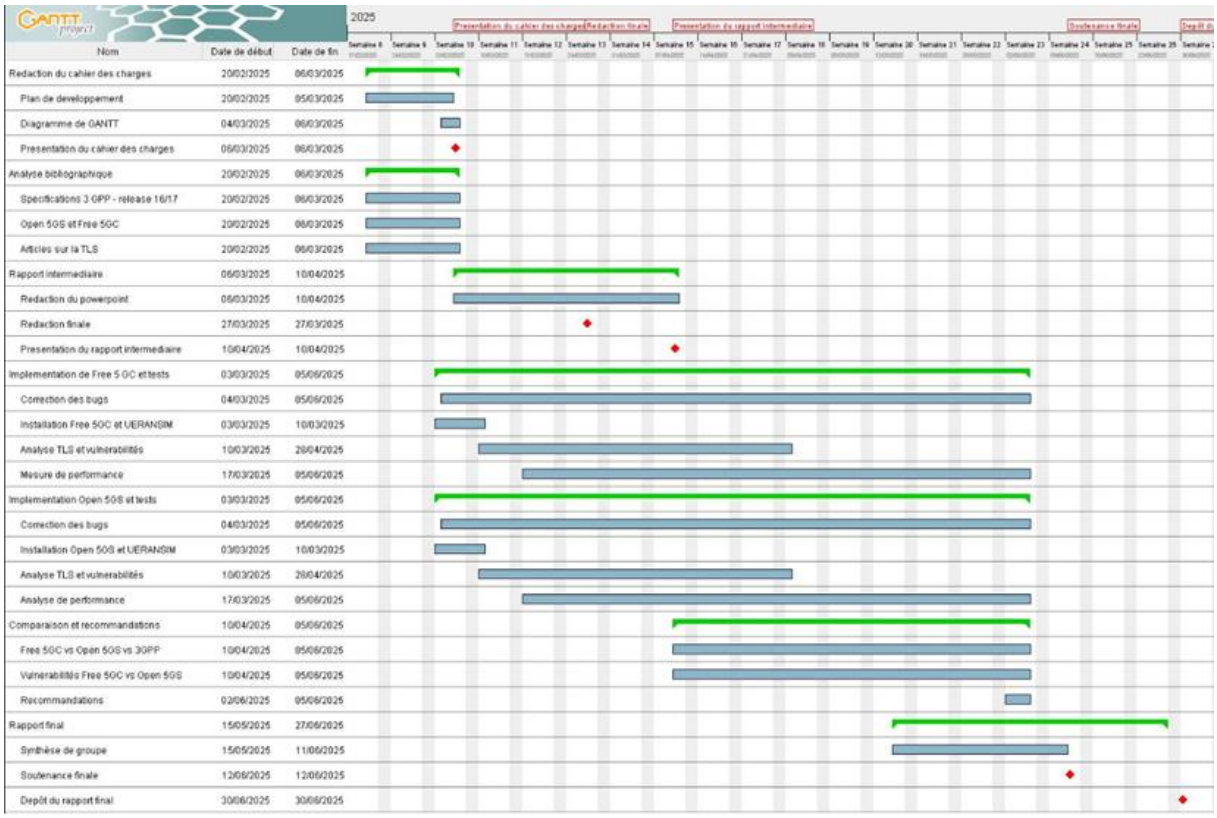
sur les performances des deux réseaux de cœur.

2.4 Recommandations et conclusion

Pour finir, nous ferons des recommandations sur la base des écarts de conformité, de vulnérabilités et de performance.

2.5 Tâches du projet – Diagramme de Gantt

Le diagramme de Gantt ci-dessous répertorie les différentes tâches du projet, les jalons ainsi que les responsables qui y sont affecté.



Nom	Date de début	Date de fin
Redaction du cahier des charges	20/02/2025	06/03/2025
Plan de developpement	20/02/2025	05/03/2025
Diagramme de GANTT	04/03/2025	06/03/2025
Presentation du cahier des charges	06/03/2025	06/03/2025
Analyse bibliographique	20/02/2025	06/03/2025
Specifications 3 GPP - release 16/17	20/02/2025	06/03/2025
Open 5GS et Free 5GC	20/02/2025	06/03/2025
Articles sur la TLS	20/02/2025	06/03/2025
Rapport intermediaire	06/03/2025	10/04/2025
Redaction du powerpoint	06/03/2025	10/04/2025
Redaction finale	27/03/2025	27/03/2025
Presentation du rapport intermediaire	10/04/2025	10/04/2025
Implementation de Free 5 GC et tests	03/03/2025	05/06/2025
Correction des bugs	04/03/2025	05/06/2025
Installation Free 5GC et UERANSIM	03/03/2025	10/03/2025
Analyse TLS et vulnerabilités	10/03/2025	28/04/2025
Mesure de performance	17/03/2025	05/06/2025
Implementation Open 5GS et tests	03/03/2025	05/06/2025
Correction des bugs	04/03/2025	05/06/2025
Installation Open 5GS et UERANSIM	03/03/2025	10/03/2025
Analyse TLS et vulnerabilités	10/03/2025	28/04/2025
Analyse de performance	17/03/2025	05/06/2025
Comparaison et recommandations	10/04/2025	05/06/2025
Free 5GC vs Open 5GS vs 3GPP	10/04/2025	05/06/2025
Vulnerabilités Free 5GC vs Open 5GS	10/04/2025	05/06/2025
Recommandations	02/06/2025	05/06/2025
Rapport final	15/05/2025	27/06/2025
Synthèse de groupe	15/05/2025	11/06/2025
Soutenance finale	12/06/2025	12/06/2025
Depôt du rapport final	30/06/2025	30/06/2025

3 Etat de l'art

3.1 Introduction à 5G SA

La 5ème génération de réseau mobile, communément appelée 5G, est le système défini par 3GPP dans les releases 15 et suivantes (Sultan, 2022). La 5G apporte des améliorations par rapport à la 4G et a été conçue notamment pour proposer une latence plus faible, des débits plus élevés, la possibilité de gérer un grand nombre de terminaux et améliorer l'expérience utilisateur quant à la qualité de service (Agiwal et al., 2016). Cela se traduit par l'introduction des nouveaux cas d'usages suivants (ITU-R, 2015) :

- « Enhanced Mobile BroadBand » (eMBB) permet d'offrir une bande passante élevée pour les services tel que la vidéo en directe ou la réalité augmentée.
- « Massive Machine-Type Communications » (mMTC) offre la possibilité de connecter un grand nombre de machines ou d'appareils notamment dans le cadre des communications de l'internet des objets (capteurs, villes intelligentes …).
- « Ultra-Reliable and Low Latency Communications » (URLLC) est prévu pour des applications critiques où la faible latence et la fiabilité du réseau sont primordiales comme les véhicules autonomes ou la télémédecine.

Pour répondre à ces usages, la 5G propose l'utilisation d'une architecture moderne qui s'oriente vers le modèle cloud dite « cloud-native » (Bolan, 2023).

On distingue deux types de déploiement 5G (ARUNRUANGSIRILERT et al., 2022). Tout d'abord, la 5G « Non Stand-Alone » ou NSA, se caractérise par l'utilisation d'un réseau d'accès radio 5G interfacé avec un cœur LTE/EPC (Long Term Evolution / Evolved Packet Core) tel qu'utilisé par les architectures 4G.

La 5G « Stand-Alone » ou SA, utilise pour sa part un réseau d'accès radio 5G avec un cœur de réseau 5G et permet alors d'adresser les différents cas d'usages cités ci-dessus.

Le réseau de cœur gère les fonctions essentielles du réseau mobile, comme la connectivité, la gestion de la mobilité, l'authentification et l'autorisation ou la gestion des données des abonnés.

Le déploiement 5G SA constitue une évolution majeure consistant à séparer le plan de contrôle du plan de trafic utilisateur. L'architecture du plan de contrôle est une architecture hybride entre des applications Cloud Native et la virtualisation dite « Service-Based Architecture » et reposant sur des fonctions de réseau virtualisées (NFV) (3GPP, 2024). C'est une architecture orienté service composée de d'agents logiciels indépendants appelés « Network Function » (NF). Ces modules exécutent une fonction spécifique et peuvent être facilement réutilisés. Ils exposent leurs services à travers des interfaces de programmation d'application API (*Architecture SBA du réseau 5G*, 2021). Chaque fonction est exécutée dans un conteneur ce qui permet de déployer et adapter dynamiquement le cœur de réseau en fonction de la charge et d'apporter rapidement de nouvelles fonctionnalités amenant donc une meilleure adaptation aux évolutions de service. Cette architecture aide également à répondre efficacement aux nouveaux cas d'usages de la 5G grâce notamment au « Network Slicing » qui permet de découper un réseau physique en plusieurs réseaux virtuels (ou "tranches"), chacun optimisé pour un usage spécifique (Xiaoyang, 2021) et exécutant son propre set de NFs.

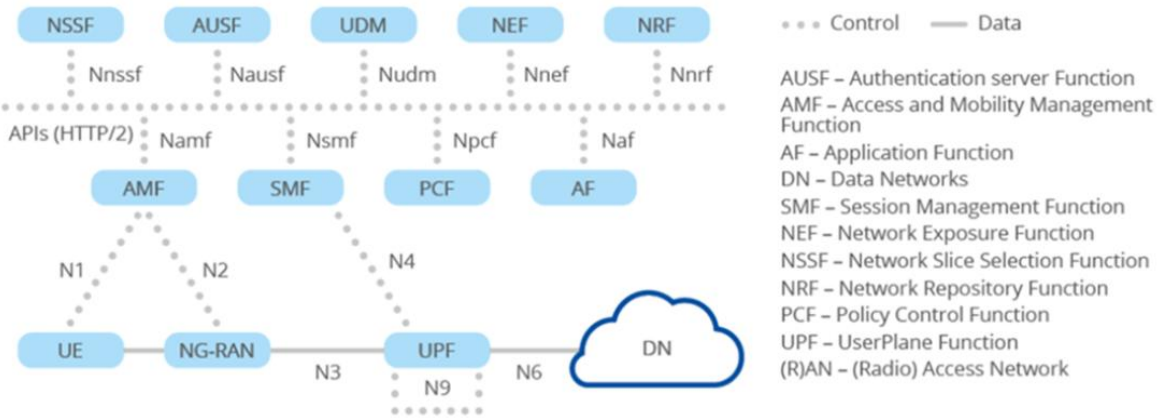


Figure 1: Service-Based Architecture - Source : European Union Agency for Cybersecurity.

La « NF Repository Function » (NRF) permet à chaque fonction de réseau de découvrir les services offerts par les autres fonctions réseau. Une fonction réseau peut être un producteur ou un consommateur de services.

Les fonctions réseau communiquent entre elles à travers de l'interface « Service Based Interface » (SBI) grâce à une API utilisant principalement le JSON et via des opérations basées sur un modèle de demande/réponse ou de souscription/notification (Brown, 2017). Le protocole HTTP/2 a été choisi par le *Core Network and Terminal specification working group 4* du 3GPP pour le transport de ces communications (Arkko, 2017).

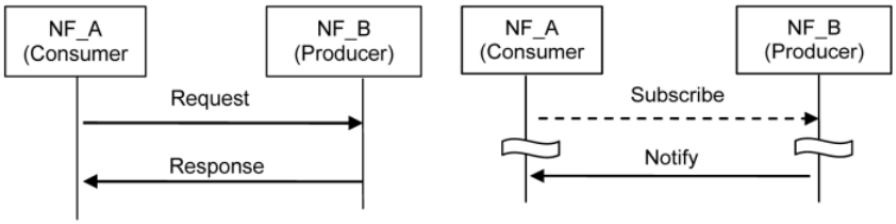


Figure 2: Types d'opérations primitives - Source : China Mobile

La sécurité de ces échanges est cruciale pour garantir la confidentialité, l'intégrité et l'authenticité des communications. Différents mécanismes sont mis en place pour assurer cette protection (Tripathi & Weissberger, 2022):

- L'authentification mutuelle des fonctions de réseau
- L'autorisation pour une fonction d'accéder aux services des autres fonctions
- Le chiffrement des communications entre les fonctions de réseau via TLS

Ces mécanismes de sécurité sont décrits dans les spécification techniques des Releases 16 et 17 de 3GPP et doivent être respectées par les constructeurs, néanmoins, le 3GPP n'impose pas leur utilisation aux opérateurs (3GPP, 2024), rendant les implémentations de cœurs 5G SA différentes et plus ou moins éloignées des spécifications 3GPP.

3.2 Authentification mutuelle des fonctions

Avant d'échanger des données, une fonction productrice d'un service et une fonction consommatrice du service, les deux NF doivent s'authentifier l'une à l'autre. Cette authentification mutuelle garantit la sécurité des communications, évite les attaques de type Man-in-the-Middle (MITM) et empêche l'usurpation d'identité d'une NF (Martins, 2023). Pour se faire on a recours à l'utilisation de mTLS (Mutual TLS). mTLS consiste en l'authentification pour chaque NF d'une autre NF par l'échange d'un certificat TLS permettant de vérifier l'identiter des NFs. La gestion de ces certificats repose sur l'exploitation d'une PKI au sein du réseau.

D'autre part, l'autorisation d'accès pour une fonction aux services des autres NFs est validée via l'utilisation du protocole OAuth 2.0. C'est une méthode bien établie pour exécuter des autorisations dans un déploiement dynamique et virtualisé. Elle repose sur un serveur d'autorisation central qui délivre des jetons d'accès aux clients (NF consommatrices de services) après authentification préalable. La NF cliente présente ensuite le jeton d'accès au producteur de services lors de l'appel d'un service. Le producteur de services NF valide d'abord le jeton d'accès avant d'accorder l'accès à ses services. Dans l'architecture SBA de 5G SA, c'est la NRF qui joue le rôle de serveur d'autorisation. Là encore c'est le protocole TLS qui est utilisé pour chiffrer l'échange du jeton bloquant ainsi son interception (Jost & Smets, 2020)

3.3 Le protocole TLS

Pour assurer l'authentification et la protection du transport entre les différentes fonctions de réseau virtualisées, le protocole TLS (Transport Layer Security) est utilisé (Jost & Smets, 2020). Ce protocole est une solution très répandue pour la protection des flux réseau. Dans ce modèle client–serveur, les données applicatives sont chiffrées de manière à assurer la confidentialité, l'intégrité et empêcher leur rejeu. Le serveur est nécessairement authentifié, et des fonctions additionnelles permettent l'authentification du client si nécessaire (ANSSI, 2020).

Depuis sa première version, TLS 1.0, parue en janvier 1999 (Allen & Dierks, 1999), l'IETF a continué le développement du protocole donnant lieu à la publication de différentes versions ; la version 1.1 parue en 2006 (Dierks & Rescorla, 2006), puis la version 1.2 en 2008 (Rescorla & Dierks, 2008) et enfin la version la plus récente, TLS 1.3 en 2018 (Rescorla, 2018).

Les différentes versions de TLS ne sont pas soumises aux mêmes vulnérabilités, et le protocole a été la cible de différentes attaques parmi lesquelles les attaques de compression comme *CRIME*, *TIME* et *BREACH*, des attaques de rétrogradation telles que *FREAK* et *LOGJAM* et des attaques sur le chiffrement en mode « Cipher Block Chaining » (CBC) telles que *BEAST*, *Lucky 13* et *POODLE* (Alemu, 2020).

Pour se prémunir au mieux de ces attaques et améliorer la sécurité du protocole au cours des différentes versions, les algorithmes de chiffrements obsolètes ont été abandonnés et le protocole a été adapté dans son fonctionnement pour le rendre plus robuste face aux vulnérabilités connues (Ali & Singh, 2023). Ainsi aujourd'hui, l'utilisation de TLS 1.0 et 1.1 est dépréciée (Moriarty & Farrell, 2021), et, bien que TLS 1.2 soit encore largement utilisé, il est recommandé l'utilisation de TLS dans sa version 1.3 (Agrawal, 2023).

Par rapport à TLS 1.2, TLS 1.3 améliore son niveau de sécurité en supprimant le support des algorithmes de chiffrement obsolètes ou jugés non sûres. Il est a noter également que le nombre de suites de chiffrement disponibles a été fortement réduit par rapport à TLS 1.2 pour ne proposer que les algorithmes les plus sécurisés, simplifiant ainsi l'usage du protocole dans sa version 1.3 (Lee et al., 2021). De plus, TLS 1.3 offre une confidentialité parfaite (Perfect Forward Secrecy-PFS) grâce à ses algorithmes d'échange de clés éphémères que sont ECDHE et DHE.

Outre ces progrès en matière de sécurité, TLS 1.3 améliore également les perfomances grâce notamment à un processu de « hadshake » optimisé, ne comportant qu'un aller retour entre le client et le serveur là ou TLS 1.2 en nécessite deux. De plus, la charge utile est désormais chiffrée le plus tôt possible pour garantir un niveau de confidentialité plus élevé. Tout message suivant le ServerHello est désormais protégé (Alemu, 2020).

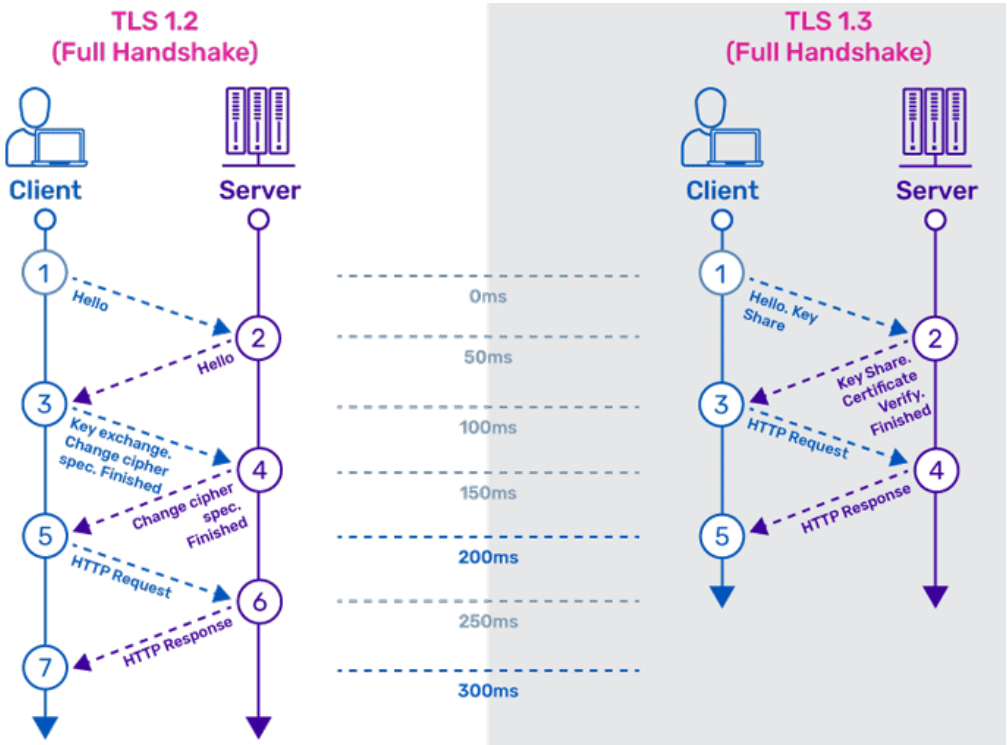


Figure 3: TLS 1.2 and 1.3 Handshake flow comparison - (« Why Is TLS 1.3 Better And Safer Than TLS 1.2? », 2025)

Il a été décrit précédemment que le cœur de réseau 5G SA, dans son architecture SBA, utilise HTTP/2 et TLS pour la communication entre ses fonctions de réseau virtualisées sur l'interface SBI. Ces communications sont donc soumises aux différentes problématiques de sécurité et de performances liées à l'utilisation du protocole TLS. C'est pourquoi, dans ses spécifications, le 3GPP donne des exigences quant à l'implémentation de TLS dans le cadre de son utilisation pour la sécurisation des communications entre les fonctions de réseau. Lorsque l'on compile les informations des releases 16 et 17, on remarque que ces exigences sont identiques dans chacune d'elles.

3.4 Les exigences de 3GPP pour l'utilisation de TLS dans le cœur 5G SA

Dans ses spécifications et notamment dans la spécification technique TS 33.210, 3GPP indique que seule l'utilisation de TLS 1.2 et 1.3 est autorisée, ainsi que DTLS 1.2 dans le cas de communications UDP (3GPP, 2020).

3.4.1 TLS 1.2

Dans le cas de TLS 1.2, une suite de chiffrement est composée des différentes parties qui indiquent les algorithmes retenus pour l'échange de clé, la signature, le chiffrement et l'authentification du message.

Figure 4 : Décomposition d'une suite de chiffrement TLS 1.2

SUITES CRYPTOGRAPHIQUES

Si le protocole TLS est utilisé dans sa version 1.2 pour le chiffrement des communications inter-NF, 3GPP stipule que les suites de chiffrements autorisées sont celles décrites dans la RFC 5246 (3GPP, 2020). La RFC 5246 donne les suites suivantes (Rescorla & Dierks, 2008) :

- TLS_RSA_WITH_AES_128_CBC_SHA (obligatoire)

Néanmoins, 3GPP apporte des précisions sur les suites qui peuvent être utilisées. Ainsi seules les suites de chiffrement avec AEAD, c'est-à-dire qui assure à la fois la confidentialité et l'intégrité et qui assurent le « perfect forward secrecy » (PFS) doivent être prises en charge.

AES CBC (Cipher Block Chaining) n'est pas un algorithme respectant AEAD. On peut citer parmi eux AES-GCM ou AES-CCM par exemple.
Pour ce qui est de PFS, DHE (Diffie-Hellman Éphémère) ou ECDHE (Elliptic Curve Diffie-Hellman Éphémère) sont conformes. La suite TLS_RSA_WITH_AES_128_CBC_SHA ne doit donc pas être utilisée.

En complément, 3GPP spécifie deux suites qui doivent obligatoirement être supportées. On notera que 3GPP recommande leur utilisation :

- TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
- TLS_DHE_RSA_WITH_AES_128_GCM_SHA256

Dans les spécifications 3GPP il est également recommandé mais non obligatoire de supporter ces deux suites cryptographiques :

- TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
- TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384

SUITE CRYPTOGRAPHIQUES AVEC CLE PRE-PARTAGEE (PSK)

Il est possible d'implémenter l'utilisation de PSK, dans ce cas les suites suivantes doivent être supportées, et leur utilisation est recommandée :

- TLS_DHE_PSK_WITH_AES_128_GCM_SHA256
- TLS_ECDHE_PSK_WITH_AES_128_GCM_SHA256

Le support de cette suite est recommandé :

- TLS_ECDHE_PSK_WITH_AES_256_GCM_SHA384

GROUPES DIFFIE-HELLMAN

Dans le cas de l'utilisation d'une suite cryptographique utilisant ECDHE (Elliptic Curve Diffie-Hellman Ephemeral), 3GPP indique le support des courbes suivantes :

- secp256r1 (obligatoire)
- secp384r1 (recommandé)
- Toutes les courbes de moins de 256 bits ne doivent pas être supportées sauf curve25519, ed25519 et W-25519

Dans le cas de l'utilisation d'une suite cryptographique utilisant DHE (Diffie-Hellman Ephemeral), 3GPP défini le support des groupes tel que :

- Le support des groupes de minimum 4096 bits est recommandé
- Le support des groupes de moins de 2048 bits ne doivent pas être supportés

ALGORITHMES DE CHIFFREMENT ET DE SIGNATURE

Les algorithmes de hachage supportés sont les suivants :

- SHA-256 (obligatoire)
- SHA-384 (recommandé)
- MD5 et SHA-1 ne doivent pas être supportés.

Pour la signature les algorithmes suivants sont supportés :

- ecdsa (obligatoire)
- rsa_pss_rsae (obligatoire)
- rsa_pkcs1 (obligatoire mais usage déconseillé)
- ecdsa_p384r1_sha384 (recommandé)

COMPRESSION TLS

Seule la méthode « NULL » doit être supportée donc pas de compression.

3.4.2 TLS 1.3

Dans le cas de TLS 1.3, les suites de chiffrement ainsi que les groupes Diffie-Hellman doivent être choisi conformément aux recommandations de la RFC 8446 de l'IETF, 3GPP y ajoute quelques compléments (3GPP, 2020).

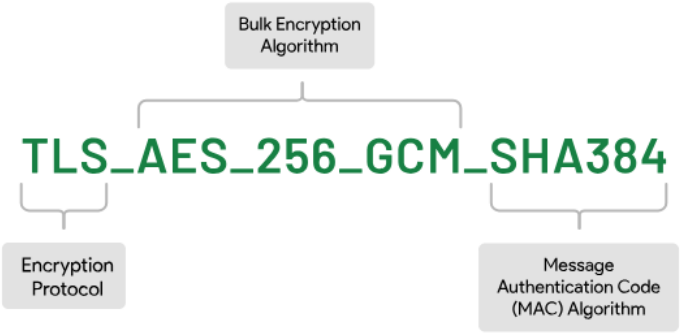


Figure 5 : Décomposition d'une suite de chiffrement TLS 1.3

SUITES CRYPTOGRAPHIQUES ET GROUPES DIFFIE-HELLMAN

La RFC 8446 définie les suites cryptographiques suivantes pour TLS 1.3 (Rescorla, 2018) :

- TLS_AES_128_GCM_SHA256 (support obligatoire)
- TLS_AES_256_GCM_SHA384 (support recommandé)
- TLS_CHACHA20_POLY1305_SHA256 (support recommandé)

Les échanges de clés doivent supporter les courbes suivantes :

- secp256r1 (RFC8446)
- secp384r1 (3GPP TS23.210 – support recommandé)

SIGNATURE TLS

Les signatures doivent supporter les algorithmes suivants :

- rsa_pkcs1_sha256 (RFC8446)
- rsa_pss_rsae_sha256 (RFC8446)
- ecdsa_secp256r1_sha256 (RFC8446)
- ecdsa_secp384r1_sha384 (3GPP TS23.210 – support recommandé)

4 Analyse

4.1 Expression des besoins

Ce projet porte sur l'analyse comparée de l'utilisation de TLS dans le plan de contrôle de cœurs de réseau 5G SA. Il consiste en une analyse documentaire des spécifications concernant l'implémentation de TLS puis au déploiement de deux environnements virtualisés fonctionnels 5G SA composés d'un cœur de réseau 5G SA (Free5GC ou Open5GS) et d'un simulateur de RAN et UE 5G (UERANSIM). Plusieurs objectifs sont ainsi définis :

- Identifier les exigences du 3GPP quant à l'utilisation de TLS comme protocole de sécurisation des communications dans le cœur 5GSA via l'étude des spécifications.
- Vérifier la conformité de l'implémentation de TLS par les deux projets de cœurs de réseau open source au regard des spécifications de 3GPP
- Mesurer l'impact de l'utilisation de TLS sur les communications inter-fonctions au sein du cœur 5G et évaluer les différences de performances entre les communications sans TLS, puis avec TLS.

4.2 Descriptif détaillé des tâches

4.2.1 Analyse des spécifications du 3GPP relatives à l'utilisation de TLS

La première tâche consiste à analyser les spécifications du 3GPP et notamment les release 16 et 17 en se focalisant sur les exigences quant à l'implémentation de TLS. 3GPP publie ses spécifications sous la forme de « Technical Specification » ou TS. Notre analyse se porte principalement sur les TS suivantes :

- TS 33.210 : Network Domain Security (NDS) ; IP network layer security
- TS 23.501 : System architecture for the 5G System (5GS)
- TS 23.502 : Procédures for the 5G System (5GS)

L'étude de ces documentations permettra de définir un référentiel des exigences dictées par le 3GPP concernant l'utilisation de TLS pour les communications inter-fonctions du plan de contrôle du cœur 5GS SA.

Ce référentiel servira de base pour comparer la manière dont est implémenté TLS dans chacun des deux cœurs de réseaux qui seront déployés et ainsi valider ou non leur conformité avec norme 3GPP.

Les principaux points de comparaison seront :

- Version de TLS utilisée
- Suite cryptographiques utilisées
- Algorithmes de chiffrement proposés.

4.2.2 Analyse préalable des documentations Free5GC et Open5GS

La solution Open5GS propose une implémentation en C des composants du cœur. Elle repose sur une architecture modulaire et elle supporte TLS via la célèbre librairie OpenSSL.

La solution Free5GC est écrit en Go est se dit conforme avec la norme 3GPP release « 15+ ». Elle utilise pour sa part la librairie crypto/tls.

Ces projets disposent d'une documentation que nous allons étudier afin de comprendre comment chacun d'eux implémente TLS pour la sécurisation des communications entre les fonctions. Il pourra être intéressant d'étudier également les documentations des librairies cryptographiques utilisées.

Cette étude documentaire devrait permettre de mettre en évidence :

- La liste de(s) version(s) de TLS proposées
- Les suites cryptographiques proposées
- Les algorithmes de chiffrement proposés.

4.2.3 Mise en place de l'environnement expérimental

Open5GS et Free5GC supportent plusieurs modes de déploiements tels que l'installation directement sur l'hôte ou la conteneurisation Docker ou Kubernetes. Nous avons choisi de déployer les cœurs de réseaux via Docker et le simulateur UERANSIM directement sur l'hôte. Nous avons effectué ce choix pour la souplesse d'implémentation et la simplicité.

Notre maquette reposera sur des machines virtuelles Ubuntu 22.04 LTS, dans un environnement VirtualBox. Les principaux composants à installer seront :

- Le cœur 5G (VM1) : Open5GS ou Free5GC, avec leurs dépendances système (MongoDB, Node.js, Go, etc.)
- Le simulateur de RAN et UE UERANSIM (VM2) qui permettra la simulation du réseau d'accès radio 5G et des terminaux compatibles 5G.
- Les outils de tests de configuration TLS : SSLyse et TLS_Scanner
- Les outils de capture et d'analyse de paquets : Wireshark

Lorsque tous les composants auront été déployés, il faudra les configurer. Ces configurations porteront principalement sur :

- Configuration réseau notamment des fonctions qui doivent communiquer en dehors du cœur.
- Configuration des interfaces SBI des fonctions pour activer ou non le chiffrement TLS en fonction des tests à réaliser
- Configuration des Mobile Country Code (MCC) / Mobile Network Code (MNC)
- Ajoute des utilisateurs dans la base
- Configuration du simulateur gNodeB et UE

4.2.4 Tests de conformité TLS et vulnérabilités des fonctions

Le but de cette tâche est de vérifier si la configuration TLS des cœurs de réseaux est conforme aux exigences du 3GPP. Pour se faire, nous utiliserons deux scanner TLS (SSLyze et TLS_Scanner) et nous analyserons chacune des fonctions du plan de contrôle afin de déterminer les caractéristiques des configurations TLS. L'utilisation de deux scanner permettra de confirmer la validité des scans en s'assurant que les résultats de chacun d'eux sont identiques. Ces résultats seront comparés au référentiel établi à la tâche 4.2.1 ce qui permettra de conclure sur le respect ou non des exigences 3GPP par ces deux cœurs open source.

Ces deux scanner proposent aussi durant leur analyse de détecter les vulnérabilités TLS connues. À partir de ces résultats, nous pourrons ensuite évaluer si les NFs sont potentiellement exposées à ces vulnérabilités dues à une erreur de configuration de TLS ou à l'utilisation d'une suite cryptographique obsolète.

4.2.5 Analyse de l'impact sur les performances

Le but ici est de déterminer l'impact de l'activation de TLS et de sa configuration sur les échanges entre les fonctions dans le plan de contrôle. Le plan de test de performances est structuré selon deux niveaux d'analyse : une analyse macro pour les procédures complètes et une analyse micro pour les échanges entre les NF durant les procédures.

Les mesures seront réalisées pour ces deux niveaux d'analyses en prenant en compte les deux configurations suivantes :

- TLS désactivé sur le plan de contrôle
- TLS activé sur les interfaces SBI des NFs

ANALYSE MACRO

En nous basant sur les procédures définies dans 3GPP TS 23.502 v17.6.0, nous avons choisi, pour réaliser les mesures, de suivre les procédures d'attachement du mobile au réseau et de l'établissement de session PDU. Lorsqu'un équipement utilisateur (UE) veut se connecter à un réseau 5G, il suit ces deux procédures. De plus elles sont simples à simuler grâce à UERANSIM.

Nous allons lancer l'enregistrement et l'établissement de session PDU pour un nombre d'UE défini évoluant par pallier, puis mesurer et analyser, dans les deux configurations ci-dessus, les métriques suivantes :

- La durée totale écoulée pour la finalisation des procédures d'enregistrement (Figure 6) pour le lot d'UE représentée par le premier message « Registration request » (0x41) du premier UE jusqu'au dernier message « Registration complete » (0x43) du dernier UE.
- La durée totale écoulée pour la finalisation des procédures d'établissement de session PDU (Figure 7) pour le lot d'UE représentée par le premier message « PDU Establishement request » (0xc1) du premier UE jusqu'au dernier message « PDU Session Establishement accept » (0xc2) du dernier UE.
- La consommation CPU/RAM de l'hôte durant ces simulations de procédures.

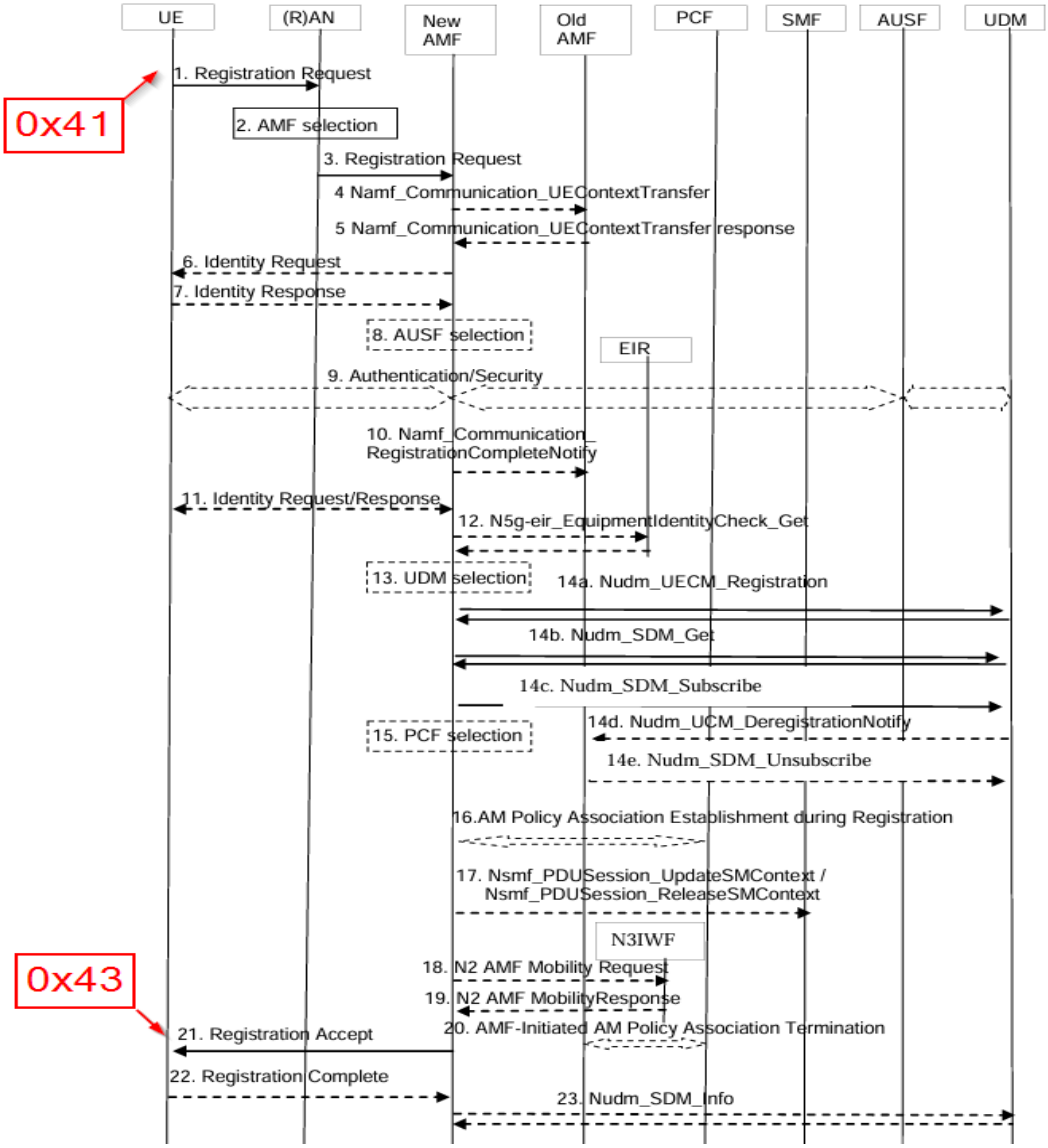


Figure 6 - Procédure d'enregistrement de l'UE

L'objectif de ces mesures et de déterminer dans quelle mesure l'activation de TLS ralenti les procédures d'enregistrement et d'établissement de session PDU et quel impact le chiffrement a-t-il sur la consommation de ressources.

Afin d'avoir un résultat représentatif, nous réalisons 10 mesures pour chaque configuration (TLS actif ou non) et pour chaque pallier de lot d'UE puis nous moyennons ces mesures que nous affichons dans un tableau comparatif.

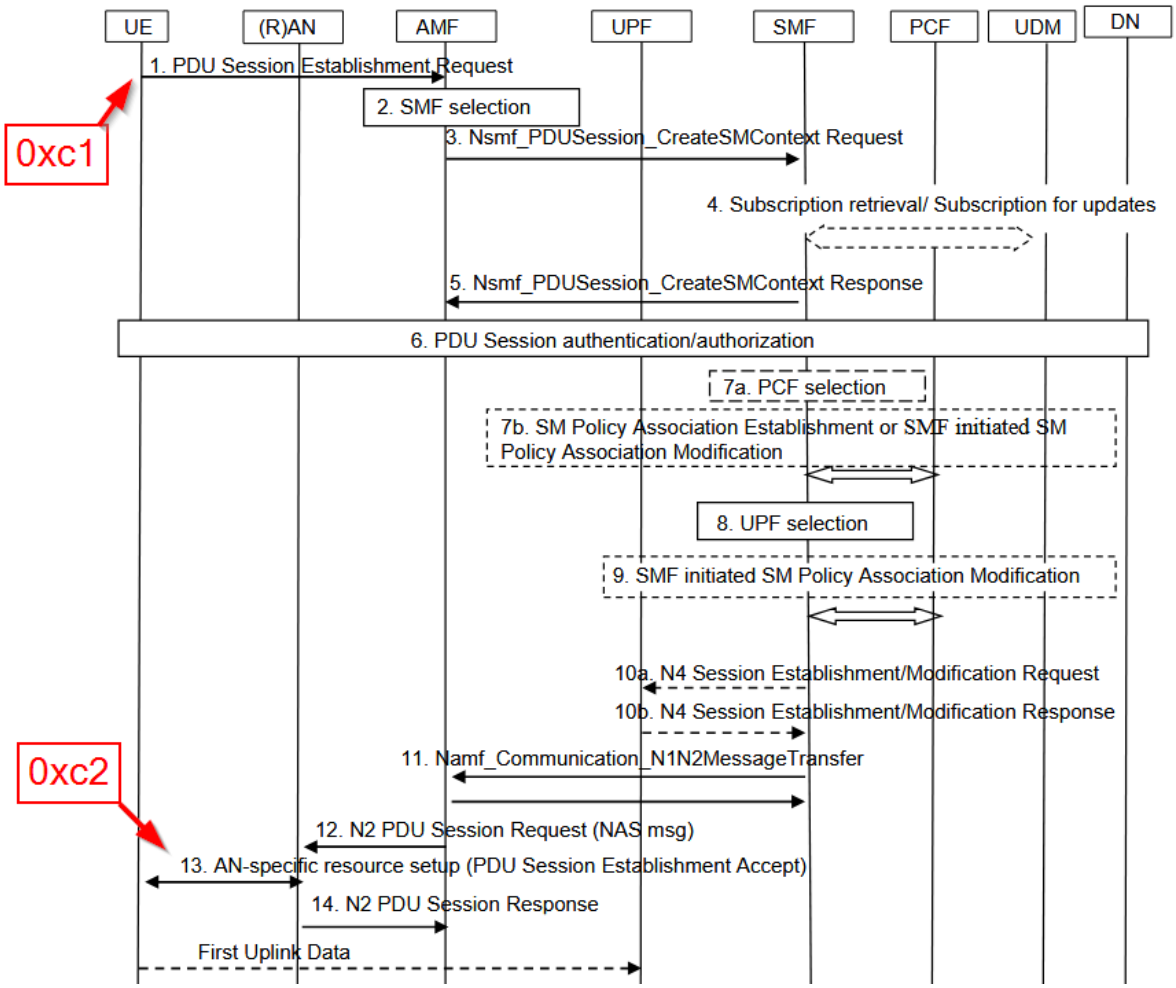


Figure 7 - Procédure d'établissement de la session PDU

ANALYSE MICRO

Ce niveau d'analyse consiste à se focaliser sur le trafic échangé entre l'AMF et l'AUSF durant la procédure d'enregistrement de l'UE et de mesurer, les différences entre les métriques suivantes :

- Nombre de paquets échangés entre les deux fonctions
- Volume de trafic échangé entre les deux fonctions

Ces mesures permettront d'analyser l'impact de TLS au niveau plus fin du paquet IP et sur le volume de trafic généré.

L'utilisation d'un outil de benchmark permettra de générer un nombre précis de message spécifique ciblant la fonction. Afin de mesurer la montée en charge de la fonction, nous mesurerons durant ce test notamment :

- La latence en millisecondes
- La perte de paquets éventuelle

4.3 Description des problèmes à résoudre

4.3.1 Mise en place l'environnement

Les multiples configurations à tester (deux cœurs, activation ou non de TLS) vont demander d'avoir un moyen efficace pour déployer les cœurs et les reconfigurer pour les adapter aux différents tests.

4.3.2 Analyse des messages chiffrés via TLS

Le rôle du protocole TLS est de chiffrer le trafic afin de le rendre illisible pour un observateur extérieur, il sera nécessaire de trouver un moyen permettant de déchiffrer les paquets afin de pouvoir déterminer certains timestamps clés pour l'analyse macro et analyser le contenu des messages pour l'analyse micro.

4.3.3 Déterminer le nombre d'UE à enregistrer

Afin de pouvoir mesurer l'évolution du comportement du cœur face à la montée en charge, nous devront déterminer la quantité d'UE qui compose le batch d'essai et les paliers de montée en charge.

4.3.4 Finesse de la mesure de la latence

La latence est un facteur critique qui conditionne directement la qualité de service, elle sera donc analysée avec la plus haute précision possible. Wireshark devrait permettre une précision à la milliseconde pour les mesures. La précision de ces mesures devra permettre d'évaluer concrètement l'impact de TLS.

5 Conception

Dans le cadre du projet, nous avons conçu et mise en œuvre un environnement expérimental qui nous a permis de mener les différents tests et mesures nécessaire à l'accomplissement des objectifs définis.

5.1 Description de l'environnement expérimental

Nous avons opté pour la virtualisation des différentes machines nécessaires à la mise en place de la maquette. Ainsi l'environnement de lab est composé de quatre machines virtuelles ayant chacune un rôle spécifique. Toujours pour des raisons de praticité de déploiement, nous avons eu recours à un hyperviseur de type 2.

L'hyperviseur utilisé est le logiciel VirtualBox d'Oracle dans sa dernière version au moment de la création de l'environnement, la version 7.1.6. L'hôte supportant cet environnement de virtualisation fonctionne sous Windows 11. La configuration matérielle de ce dernier est la suivante :

- CPU : AMD Ryzen 7 7800X3D (8 cores/16 threads à 4.2 Ghz)
- 64 Go de RAM DDR5 6800Mhz
- Stockage 2x SSD NVM-e PCI-e 5.0 (lecture/écriture : 6600Mo/s / 5500 Mo/s)
- Carte réseau Intel X550 10G

Toutes les machines virtuelles ont été déployées avec le système d'exploitation Linux Ubuntu en version 22.04.02 LTS. Les différentes VMs portent les rôles suivants :

- VM1 – Cœur Free5GC : Cette machine héberge le cœur 5G open source Free5GC dans sa version Docker. Elle héberge également les conteneurs Wireshark, SSLyze et TLS-Scanner. Sa configuration est telle que :
  - 3 vCPU
  - 8 Go de RAM
  - 100 Go de Stockage
  - 1 carte réseau
- VM2 – Cœur Open5GS : Cette machine héberge le cœur 5G open source Open5GS dans sa version Docker et les outils Wireshark, SSLyze et TLS-Scanner. Sa configuration est telle que :
  - 3 vCPU
  - 8 Go de RAM
  - 100 Go de Stockage
  - 1 carte réseau

Bien qu'il aurait été possible d'utiliser la même machine pour les deux cœurs, nous avons choisi de séparer ces derniers sur deux VMs distinctes pour des questions de praticité pour les tests. D'autre part, lors des tests de performances, seule une VM sur les deux est démarrée afin d'éviter de maximiser les ressources disponibles.

- VM3 – UERANSIM : Cette machine héberge le simulateur de réseau radio et d'UE. Sa configuration est la suivante :
  - 3 vCPU
  - 8 Go de RAM
  - 100 Go de Stockage
  - 1 carte réseau
- VM4 – Supervision : Cette dernière machine héberge les outils de supervision Grafana et Prometheus permettant la collecte de métriques de consommation RAM et CPU des VM de cœur 5G. Elle possède les caractéristiques suivantes :
  - 2 vCPU
  - 8 Go de RAM
  - 250 Go de Stockage
  - 1 carte réseau

Les différents cœurs 5G open source à tester ainsi que les divers outils nécessaires sont déployés sur ces VM Linux via la conteneurisation Docker, qui est une solution à empreinte légère, peu exigeante en ressources CPU et mémoire. Docker permet de virtualiser un environnement d'exécution pour une application donnée et lui permet de s'exécuter de manière isolée. Il offre la possibilité d'utiliser des images pré-packagées et d'en créer un ou plusieurs containers selon le besoin. Docker est déployé dans sa version 28.0.4 sur les machines.

```
alex@open5gs:~$ sudo docker version
Client: Docker Engine - Community
 Version:           28.0.4
 API version:       1.48
 Go version:        go1.23.7
 Git commit:        b8034c0
 Built:             Tue Mar 25 15:07:11 2025
 OS/Arch:           linux/amd64
 Context:           default
alex@open5gs:~$ docker compose version
Docker Compose version v2.34.0
```

Figure 8 : Docker installé en v. 28.0.4 et Docker Compose en v2.34.0

Docker Compose est l'orchestrateur de conteneurs permettant d'automatiser la création, le déploiement et la mise en réseau des containers. Grâce à Docker Compose, nous pouvons redéployer facilement les différentes fonctions des deux cœurs de réseau 5G dans les différentes configurations nécessaires aux tests. Pour cela différents fichiers compose sont utilisés, faisant appel aux fichiers de configurations des cœurs 5GS activant ou non TLS, permettant de déployer toutes les fonctions du cœur, dans leur configuration avec ou sans TLS via une seule commande. Docker Compose est utilisée dans sa version 2.34.0.

```
docker compose -f docker-compose-tls.yaml up -d
docker compose -f docker-compose-tls.yaml down
```

Figure 9 - Commandes permettant de démarrer et d'arrêter les NFs de Free5GC avec TLS activé

Finalement, l'architecture cible de l'environnement expérimental peut être représentée par le schéma ci-dessous :

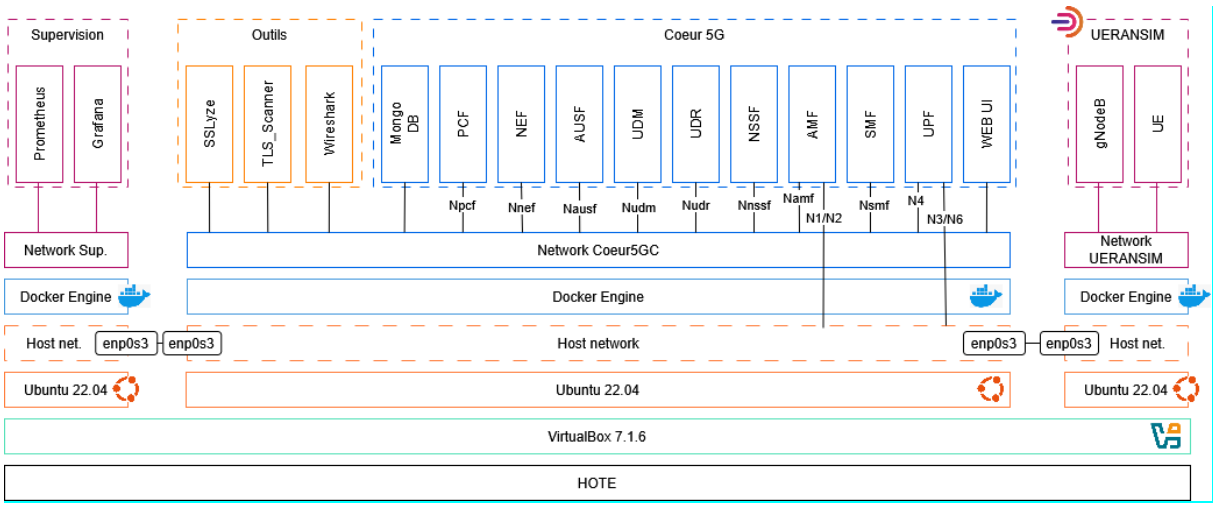


Figure 10 - Schéma d'architecture de l'environnement expérimental

5.2 Logiciels et outils utilisés

5.2.1 Cœurs 5G open sources et simulateur

Free5GC est un réseau de cœur 5G open source écrit en Go et sous licence Apache 2.0. Son architecture repose sur des micro-services, où chaque fonction de réseau joue un rôle tel que définies dans les spécifications 3GPP pour le plan de contrôle. Il utilise la librairie crypto/tls pour les fonctionnalités de sécurisation.

- Free5GC est déployé en version 4.0.1.

Le second cœur de réseau à tester est Open5GS qui est également un projet open source, il est écrit en C et est distribué sous licence GPLv2. La librairie SSL utilisée par ce projet est OpenSSL. Il peut également s'exécuter dans un environnement conteneurisé. Il offre une interface utilisateur pour piloter l'architecture.

- Open5GS est déployé en version 2.7.5.

UERANSIM est un simulateur d'équipement utilisateur (UE) et de station radio de base (gNodeB), et peut très bien interopérer avec Open5GS et Free5GC, son rôle dans cette analyse est de déclencher les procédures telle que l'enregistrement d'un mobile dans le réseau ce qui nous permet de capturer et analyser les échanges au sein du cœur 5G pendant ces procédures.

- UERANSIM est déployé dans sa version 3.2.7.

5.2.2 Outils de tests et d'analyse

Afin de valider la conformité TLS des différents cœurs 5G SA, nous réalisons un scan des fonctions du plan de contrôle. Pour cela, nous utilisons les deux scanner TLS suivants :

- **TLS-Scanner** qui est un outil développé par des chercheurs de la Ruhr-Universität Bochum qui permet de vérifier la configuration de SSL/TLS sur le serveur cible, en collectant un ensemble d'informations telles que les certificats X.509 échangés, les versions de TLS supportées par le serveur et les primitives cryptographiques utilisées ainsi que leur ordre de proposition par le serveur. Il permet également d'évaluer les vulnérabilités.
  Pour se faire, TLS-Scanner se fait passer pour un client en initiant une connexion au serveur.
  - ➢ La version utilisée pour ce projet est la 6.1.6.
- **SSLyze** est un autre outil open source créé par nabla-c0de écrit en Python qui possède des fonctionnalités similaires à TLS-Scanner.
  - ➢ Il sera utilisé dans sa version 6.1.0.

```
* SSL 2.0 Cipher Suites:
    Attempted to connect using 7 cipher suites; the server rejected all cipher suites.

* SSL 3.0 Cipher Suites:
    Attempted to connect using 80 cipher suites; the server rejected all cipher suites.

* TLS 1.0 Cipher Suites:
    Attempted to connect using 80 cipher suites; the server rejected all cipher suites.

* TLS 1.1 Cipher Suites:
    Attempted to connect using 80 cipher suites; the server rejected all cipher suites.

* TLS 1.2 Cipher Suites:
    Attempted to connect using 156 cipher suites.

    The server accepted the following 3 cipher suites:
       TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256       256       ECDH: prime256v1 (256 bits)
       TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384             256       ECDH: prime256v1 (256 bits)
       TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256             128       ECDH: prime256v1 (256 bits)

    The group of cipher suites supported by the server has the following properties:
      Forward Secrecy                    OK - Supported
      Legacy RC4 Algorithm               OK - Not Supported


* TLS 1.3 Cipher Suites:
    Attempted to connect using 5 cipher suites.

    The server accepted the following 3 cipher suites:
       TLS_CHACHA20_POLY1305_SHA256                      256       ECDH: prime256v1 (256 bits)
       TLS_AES_256_GCM_SHA384                            256       ECDH: prime256v1 (256 bits)
       TLS_AES_128_GCM_SHA256                            128       ECDH: prime256v1 (256 bits)
```

Figure 11 : Exemple de résultat d'un scan réalisé via SSLyze

Les deux scanner sont utilisé pour scanner les fonctions, ce qui permet de corroborer les résultats et ainsi valider l'analyse.

Un second aspect du projet nécessite de capturer le trafic échangé entre les fonctions de réseau. Pour se faire, nous utiliserons Wireshark dans sa version 4.4.5. Wireshark est un analyseur de paquets réseau open-source qui permet de capturer, inspecter et analyser le trafic réseau. Il permet de réaliser un filtrage avancé, l'analyse protocolaire et la reconstruction et le

suivi des flux.

Wireshark est utilisable en ligne de commande via une interface graphique pratique pour l'analyse.

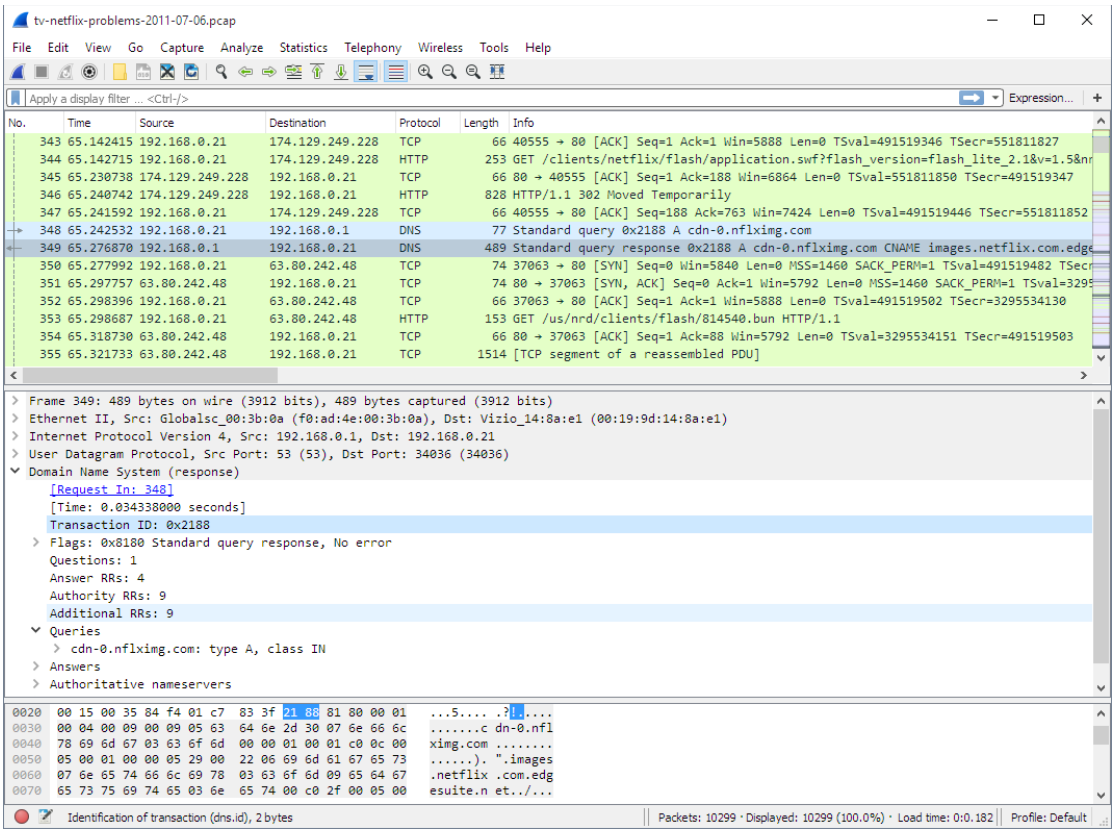


Figure 12 : Interface graphique de Wireshark

5.3 Configuration de l'environnement

La mise en place de l'environnement expérimental nécessite plusieurs étapes qui sont les suivantes :

- Création et configuration des 4 machines virtuelles dans l'hyperviseur
- Installation et mise à jour de Ubuntu 22.04.05 LTS
- Configuration de base des VMs (adresse IP, DNS, NTP…)
- Installation de Docker et Docker Compose sur ces machines.
- Téléchargement depuis les dépôts GitHub des différents projets nécessaires sur leur machines respectives
- Installation des outils annexes depuis Docker Hub ou dépôt GitHub
- Préparation des fichiers de configurations du simulateur UERANSIM et des cœurs 5G
- Enregistrement des utilisateurs dans les bases des cœurs 5G
- Préparation des fichiers Docker Compose pour les cœurs 5G

Les tâches de configuration de base des machines virtuelles ne sont pas développées ici car elles sont tout à fait standard et peu intéressantes dans le contexte.

Les principales configurations portent sur les fonctions du cœur de réseau 5G et plus particulièrement la configuration TLS des interfaces SBI ainsi que sur la configuration du simulateur de terminaux et de réseau radio.

5.3.1 Configuration du simulateur UERANSIM

UERANSIM comporte deux fichiers de configuration principaux. Il est a noté que des templates pour Open5GS et Free5GC sont fournis dans le GitHub UERANSIM (répertoire *config/*).

- **Fichier de configuration de l'UE** : Il est nécessaire de modifier :
  - Le supi : n° IMSI de l'UE, seul IMSI est supporté par UERANSIM
  - Les codes MCC et MNC pour qu'ils correspondent à l'environnement de cœur 5G
  - Les différentes clés d'associations au réseau sont déjà définies via le template en fonction du cœur concerné.

```
# IMSI number of the UE. IMSI = [MCC|MNC|MSISDN] (In total 15 digits)
supi: 'imsi-001010000000001'
# Mobile Country Code value of HPLMN
mcc: '001'
# Mobile Network Code value of HPLMN (2 or 3 digits)
mnc: '01'
# SUCI Protection Scheme : 0 for Null-scheme, 1 for Profile A and 2 for Profile B
protectionScheme: 0
# Home Network Public Key for protecting with SUCI Profile A
homeNetworkPublicKey: '5a8d38864820197c3394b92613b20b91633cbd897119273bf8e4a6f4eec0a650'
# Home Network Public Key ID for protecting with SUCI Profile A
homeNetworkPublicKeyId: 1
# Routing Indicator
routingIndicator: '0000'

# Permanent subscription key
key: '465B5CE8B199B49FAA5F0A2EE238A6BC'
# Operator code (OP or OPC) of the UE
op: 'E8ED289DEBA952E4283B54E88E6183CA'
# This value specifies the OP type and it can be either 'OP' or 'OPC'
opType: 'OPC'
# Authentication Management Field (AMF) value
amf: '8000'
# IMEI number of the device. It is used if no SUPI is provided
imei: '356938035643803'
# IMEISV number of the device. It is used if no SUPI and IMEI is provided
imeiSv: '4370816125816151'
```

Figure 13 : Fichier de configuration UERANSIM - open5gs-ue.yaml

- **Fichier de configuration du gNodeB** : la configuration consiste notamment à :
  - Modifier les codes MCC/MNC
  - Indiquer les adresses IP des interfaces locales lien radio, N2 et N3
  - L'adresse IP de la fonction AMF du cœur (N2)

```
mcc: '001'          # Mobile Country Code value
mnc: '01'           # Mobile Network Code value (2 or 3 digits)

nci: '0x000000010'  # NR Cell Identity (36-bit)
idLength: 32        # NR gNB ID length in bits [22...32]
tac: 1              # Tracking Area Code

linkIp: 192.168.45.68   # gNB's local IP address for Radio Link Simulation (Usually same with local IP)
ngapIp: 192.168.45.68   # gNB's local IP address for N2 Interface (Usually same with local IP)
gtpIp: 192.168.45.68    # gNB's local IP address for N3 Interface (Usually same with local IP)

# List of AMF address information
amfConfigs:
  - address: 192.168.45.19
    port: 38412

# List of supported S-NSSAIs by this gNB
slices:
  - sst: 1

# Indicates whether or not SCTP stream number errors should be ignored.
ignoreStreamIds: true
```

Figure 14 : Fichier de configuration UERANSIM - open5gs-gnb.yaml

5.3.2 Configuration de Free5GC

La configuration de Free5GC consiste à préparer deux jeux de fichiers de configuration YAML pour les différentes fonctions. Un de ces jeu contient les fichiers de configuration des fonctions sans TLS, l'autre les fichiers de configuration avec TLS activé. Il suffit de faire référence à l'un ou l'autre de ces jeux de fichiers dans le fichier Compose pour lancer le cœur avec TLS activé sur les SBI ou non.

```
alex@f5gc:~/free5gc-compose-2VM$ ls config-TLS/
amfcfg.yaml   n3iwfcfg.yaml   nrfcfg.yaml   tngfcfg.yaml     uecfg.yaml      upf-iptables.sh
ausfcfg.yaml  n3iwf-ipsec.sh  nssfcfg.yaml  udmcfg.yaml      uerouting.yaml  webuicfg.yaml
chfcfg.yaml   n3uecfg.yaml    pcfcfg.yaml   udrcfg.yaml      ULCL
gnbcfg.yaml   nefcfg.yaml     smfcfg.yaml   uecfg-ulcl.yaml  upfcfg.yaml
alex@f5gc:~/free5gc-compose-2VM$ ls config
amfcfg.yaml   n3iwfcfg.yaml   nrfcfg.yaml   tngfcfg.yaml     uecfg.yaml      upf-iptables.sh
ausfcfg.yaml  n3iwf-ipsec.sh  nssfcfg.yaml  udmcfg.yaml      uerouting.yaml  webuicfg.yaml
chfcfg.yaml   n3uecfg.yaml    pcfcfg.yaml   udrcfg.yaml      ULCL
gnbcfg.yaml   nefcfg.yaml     smfcfg.yaml   uecfg-ulcl.yaml  upfcfg.yaml
```

Fichiers avec TLS

Fichiers sans TLS

Figure 15 : Fichiers de configuration des NFs dans le dossier config/ et config-TLS/

```
GNU nano 6.2                    config/amfcfg.yaml
info:
  version: 1.0.9
  description: AMF initial local configuration

configuration:
  amfName: AMF # the name of this AMF
  ngapIpList:  # the IP list of N2 interfaces on this AMF
    - 10.100.210.16
  ngapPort: 38412 # the SCTP port listened by NGAP
  sbi: # Service-based interface information
    scheme: http # the protocol for sbi (http or https)
    registerIPv4: amf.free5gc.org # IP used to register to NRF
    bindingIPv4: amf.free5gc.org  # IP used to bind the service
```

TLS off

```
GNU nano 6.2                    config-TLS/amfcfg.yaml
info:
  version: 1.0.9
  description: AMF initial local configuration

configuration:
  amfName: AMF # the name of this AMF
  ngapIpList:  # the IP list of N2 interfaces on this AMF
    - 10.100.210.16
  ngapPort: 38412 # the SCTP port listened by NGAP
  sbi: # Service-based interface information
    scheme: https # the protocol for sbi (http or https)
    registerIPv4: amf.free5gc.org # IP used to register to NRF
    bindingIPv4: amf.free5gc.org  # IP used to bind the service
```

TLS on

Figure 16 : Extrait des fichiers de configuration de la fonction AMF : config/amfcfg.yaml et config-TLS/amfcfg.yaml

```
GNU nano 6.2                    docker-compose.yaml
free5gc-amf:
  container_name: amf
  image: free5gc/amf:v4.0.1
  command: ./amf -c ./config/amfcfg.yaml
  expose:
    - "8000"
    - "38412:38412/sctp"
  volumes:
    - ./config/amfcfg.yaml:/free5gc/config/amfcfg.yaml
    - ./cert:/free5gc/cert
```

TLS off

```
GNU nano 6.2                    docker-compose-tls.yaml
free5gc-amf:
  container_name: amf
  image: free5gc/amf:v4.0.1
  command: ./amf -c ./config/amfcfg.yaml
  expose:
    - "8000"
    - "38412:38412/sctp"
  volumes:
    - ./config-TLS/amfcfg.yaml:/free5gc/config/amfcfg.yaml
    - ./cert:/free5gc/cert
```

TLS on

Figure 17 : Extrait des fichier docker-compose.yaml et docker-compose-tls.yaml. Cette partie est responsable du provisionnement du conteneur de l'AMF. Il fait appel au fichier de configuration avec ou sans TLS.

5.3.3 Configuration d'Open5GS

Comme pour Free5GC, la logique est identique, deux jeux de fichiers de configuration sont créés et deux fichiers Compose sont appelés en fonction de la configuration de TLS voulue.

```
alex@o5gs-Docker:~/docker-open5gs$ ls configs/basictls/
amf.yaml   bsf.yaml   nssf.yaml   smf.yaml   udr.yaml
ausf.yaml  nrf.yaml   pcf.yaml    udm.yaml   upf.yaml
alex@o5gs-Docker:~/docker-open5gs$ ls configs/basic/
amf.yaml   bsf.yaml   nssf.yaml   smf.yaml   udr.yaml
ausf.yaml  nrf.yaml   pcf.yaml    udm.yaml   upf.yaml
```

Fichiers avec TLS

Fichiers sans TLS

Figure 18 - Fichiers de configuration des NFs dans le dossier configs/basic/ et configs/basictls/

```
GNU nano 6.2                    configs/basic/amf.yaml
amf:
  sbi:
    server:
      - address: amf.open5gs.org
        port: 80
    client:
      nrf:
        - uri: http://nrf.open5gs.org:80
  ngap:
    server:
      - address: amf.open5gs.org
```

TLS off

```
GNU nano 6.2                    configs/basictls/amf.yaml
default:
  tls:
    server:
      scheme: https
      private_key: /etc/open5gs/tls/amf.key
      cert: /etc/open5gs/tls/amf.crt
      sslkeylogfile: /var/log/open5gs/tls/amf-server-sslkeylog.>
    client:
      scheme: https
      cacert: /etc/open5gs/tls/ca.crt
      client_sslkeylogfile: /var/log/open5gs/tls/amf-client-ssl>
sbi:
  server:
    - address: amf.localdomain #.open5gs.org
  client:
    nrf:
      - uri: https://nrf.localdomain #.open5gs.org
```

TLS on

Figure 19 - Extrait des fichiers de configuration de l'AMF : configs/basic/amf.yaml et configs/basictls/amf.yaml

```
GNU nano 6.2                    basic/docker-compose.yaml
configs:
  nrf_config:
    file: ../../configs/basic/nrf.yaml
  ausf_config:
    file: ../../configs/basic/ausf.yaml
  udm_config:
    file: ../../configs/basic/udm.yaml
```

TLS off

```
GNU nano 6.2                    basictls/docker-compose.yaml
configs:
  nrf_config:
    file: ../../configs/basictls/nrf.yaml
  ausf_config:
    file: ../../configs/basictls/ausf.yaml
  udm_config:
    file: ../../configs/basictls/udm.yaml
```

TLS on

Figure 20 - Extrait des fichier docker-compose.yaml. Ce bloc indique les sources des fichiers de configurations pour provisionner les conteneurs

6 Compte rendu du projet

6.1 Déroulement du projet

L'ensemble des phases prévues au plan de développement ont pu être finalisées. Le protocole de tests sur l'aspect performance a été revu à la suite des diverses avancées du projet. Le cahier des charges prévu initialement a pu être respecté et les trois objectifs ont été atteints. La mise en place de l'environnement a été faite et a permis de dérouler le protocole de tests prévu dans les différentes configurations du cœur, avec TLS activé ou non.

Dans ce chapitre, nous présentons les résultats obtenus après analyse des différentes mesures réalisées.

6.2 Résultats obtenus

6.2.1 Etude de la conformité TLS

Les études documentaires décrites précédemment ainsi que le scan des différentes fonctions de réseau des deux cœurs 5G open source ont permis de déterminer leurs implémentations de TLS et leur niveau de conformité en regard des exigences de 3GPP. Les résultats, présentées dans les tableaux ci-dessous, montrent pour chacun des cœurs, les versions de TLS et les suites cryptographiques supportées, la légende permet comparer ces résultats aux exigences de 3GPP.

Nous avons scanner toutes les fonctions du cœur de réseau afin de confirmer que la configuration TLS est homogène pour toutes les NFs, ce qui est le cas pour Free5GC comme pour Open5GS.

Le scan des NFs de Free5GC montre que les versions 1.0, 1.1, 1.2 et 1.3 de TLS sont supportées. 3GPP indique cependant que les 1.0 et 1.1 ne doivent pas être supportées. D'autre part, ces versions sont officiellement dépréciées par l'IETF (Moriarty & Farrell, 2021). Le support par Free5GC de ces deux versions de TLS peut présenter un problème de sécurité.

Version TLS	Refusées		SSL 2.0 SSL 3.0
	Acceptées		TLS 1.0 ⊖ TLS 1.1 ⊖ TLS 1.2 TLS 1.3
Suites crypto.	Acceptées	TLS 1.2	TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA ✕ TLS_RSA_WITH_AES_128_GCM_SHA256 ✕ TLS_RSA_WITH_AES_256_GCM_SHA384 ✕ TLS_RSA_WITH_AES_256_CBC_SHA ✕ TLS_RSA_WITH_AES_128_CBC_SHA ✕ TLS_RSA_WITH_3DES_EDE_CBC_SHA ✕ TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA ✕ TLS_ECDHE_RSA_WITH_3DES_EDE_CBC_SHA ✕
		TLS 1.3	TLS_AES_256_GCM_SHA384 TLS_CHACHA20_POLY1305_SHA256 TLS_AES_128_GCM_SHA256
	Manquante	TLS 1.2	TLS_DHE_RSA_WITH_AES_128_GCM_SHA256 TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 TLS_DHE_PSK_WITH_AES_128_GCM_SHA256 (si PSK) TLS_ECDHE_PSK_WITH_AES_128_GCM_SHA256 (si PSK) TLS_ECDHE_PSK_WITH_AES_256_GCM_SHA384 (si PSK)

Légende :
- Exigences 3GPP :
  - Support obligatoire
  - Support recommandé
  - Ne doit pas être supporté
  - Support non défini
- Symboles :
  - ⊖ version obsolète
  - ✕ suite faible

Figure 21 - Résultats du scan TLS des fonctions de Free5GC

D'autre part, on constate que Free5GC supporte 14 suites cryptographiques. Le support est obligatoire pour deux de ces suites et recommandé pour trois autres. Une suite est quant à elle supportée alors qu'elle est absente des spécifications 3GPP, néanmoins cette suite est sûre comme indiqué dans l'Annexe B – Sécurité des suites cryptographiques.

En outre, Free5GC implémente huit suites qui ne font pas partie des spécifications de 3GPP, et pour cause, elles sont considérées comme faibles car utilisant des algorithmes vulnérables tels 3DES ou SHA-1, 3GPP spécifie de plus que ce dernier ne doit pas être supporté. Certaines suites utilisent l'algorithme CDC (Cipher Block Chaining) ce qui n'assure pas le « perfect forward secrecy » pourtant obligatoire selon 3GPP.

Comme pour les versions de TLS, l'utilisation de suites de chiffrement faibles pourrait avoir un impact sur la sécurité du cœur 5G.

Le dernier point à noter concerne les suites manquantes. En effet, en comparant les exigences de 3GPP aux résultats du scan, 5 suites dont le support est obligatoire ou recommandé sont absente des suites proposées par Free5GC. Cela concerne notamment les suites permettant l'utilisation de clés pré-partagées (PSK).

Dans le cas d'Open5GS, on constate que seules les versions 1.2 et 1.3 de TLS sont supportées ce qui est conforme aux exigences de 3GPP.

D'autre part, Open5GS ne supporte que six suites de chiffrements parmi lesquelles deux dont le support est obligatoire au regard de exigences 3GPP et trois dont le support est

recommandé. Open5GS supporte également une suite supplémentaire non proposée par 3GPP.

Enfin, on constate qu'Open5GS ne supporte pas 3 des suites dont 3GPP exige l'implémentation, deux de ces suites étant par ailleurs réservée à l'usage avec clé pré-partagée. Les deux dernières suites non supportées sont des suites dont le support est recommandé par 3GPP mais non obligatoire.

Open5GS			
Version TLS	Refusées		SSL 2.0 SSL 3.0 TLS 1.0 TLS 1.1
	Acceptées		TLS 1.2 TLS 1.3
Suites crypto.	Acceptées	TLS 1.2	TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256
		TLS 1.3	TLS_AES_256_GCM_SHA384 TLS_CHACHA20_POLY1305_SHA256 TLS_AES_128_GCM_SHA256
	Manquante	TLS 1.2	TLS_DHE_RSA_WITH_AES_128_GCM_SHA256 TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 TLS_DHE_PSK_WITH_AES_128_GCM_SHA256 (si PSK) TLS_ECDHE_PSK_WITH_AES_128_GCM_SHA256 (si PSK) TLS_ECDHE_PSK_WITH_AES_256_GCM_SHA384 (si PSK)

Légende :
- Exigences 3GPP :
  - Support obligatoire
  - Support recommandé
  - Ne doit pas être supporté
  - Support non défini
- Symboles :
  - ⊖ version obsolète
  - ✕ suite faible

Figure 22 - Résultats du scan TLS des fonctions d'Open5GS

Les scans TLS des deux cœurs 5G nous montrent qu'aucun des deux cœurs ne respecte l'intégralité des exigences de 3GPP. Free5GC implémente des versions et suites obsolètes qui présentent un risque de sécurité sérieux en cas de compromission. Néanmoins, un point positif est que Free5GC utilise l'ordonnancement des suites, ce qui permet lorsque le client et le serveur sont compatibles d'utiliser prioritairement TLS 1.3. C'est d'ailleurs le cas dans notre maquette. Open5GS quant à lui est plus sécurisé dans son implémentation grâce notamment à la limitation des versions de TLS supportées et du nombre de suites disponibles qui exclut toute suite faible, mais il manque certaines suites cryptographiques pourtant obligatoires selon 3GPP. On retrouve également ces manques chez Free5GC. Il est intéressant de remarquer que si Free5GC supprimait le support de versions et suite obsolètes, alors les deux cœurs 5G proposeraient la même configuration TLS.

6.2.2 Analyse de l'impact de TLS sur les performances

L'activation de TLS sur les interfaces des fonctions du cœur à un impact sur les performances du cœur et sa consommation de ressources qui sont analysés dans les chapitres suivants.

ANALYSE MACRO – IMPACT GENERAL

Les captures de trafic réalisées pendant l'analyse macro permet de mettre en évidence l'impact général de TLS sur les performances du cœur et sa consommation de ressources. Comme vu au chapitre 4.2.5, une série de 10 mesures est réalisée pour chaque configuration et nous réalisons une moyenne pour les comparer.

Nous présentons tout d'abord les résultats du projet Free5GC. La Figure 23 ci-dessous présente la distribution des données pour les différents lots de mesures.

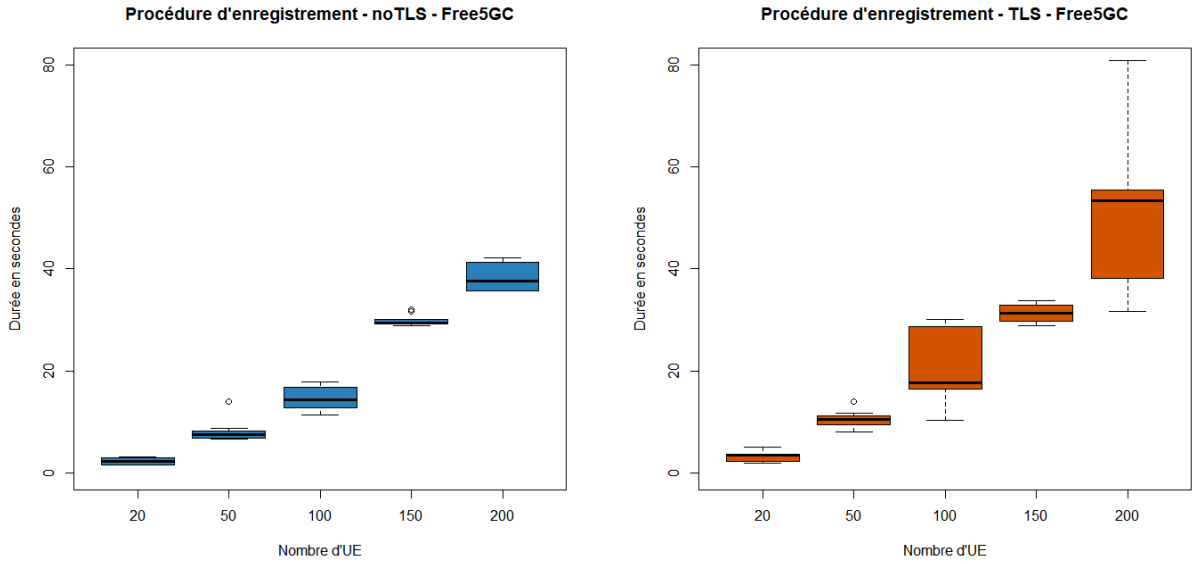


Figure 23 - Répartition des données de mesure - Procédures d'enregistrement - Free5GC

Le Tableau 1 ci-dessous présente la moyenne des 10 mesures obtenues dans les deux cas étudiés, pour différents lots d'UE réalisant les procédures ainsi que la consommations moyenne en RAM et CPU de l'hôte lors de ces tests.

On remarque que la durée totale nécessaire pour mener à terme les procédures d'enregistrement des UE augmente fortement à mesure que la charge augmente. L'activation de TLS sur les fonctions fait augmenter le temps nécessaire pour réaliser les procédures de 30% à 40%.

free5GC	TLS off			TLS on			Delta en % NoTLS vs. TLS		
Nombre d'UE	Durée moyenne de l'enregistrement	Conso CPU moyenne	Conso RAM moyenne	Durée moyenne de l'enregistrement	Conso CPU moyenne	Conso RAM moyenne	Temps de l'opération	Conso CPU moyenne	Conso RAM moyenne
20	2,325 secondes	24,77 %	1,76 GB	3,326 secondes	28,87 %	2,48 GB	+43,02 %	+15,51 %	+1,77 %
50	8,175 secondes	33,73 %	2,29 GB	10,429 secondes	44,31 %	2,48 GB	+27,57 %	+31,37 %	+8,25 %
100	14,711 secondes	39,71 %	2,31 GB	20,782 secondes	47,74 %	2,61 GB	+41,27 %	+20,12 %	+13,04 %
150	29,892 secondes	44,73 %	2,33 GB	31,365 secondes	51,34 %	2,99 GB	+4,93 %	+14,78 %	+28,67 %
200	38,437 secondes	52,47 %	2,35 GB	51,196 secondes	59,75 %	3,14 GB	+33,19 %	+13,87 %	+33,7 %

Tableau 1 - Analyse de la durée des procédures d'enregistrement d'UE avec et sans TLS - Cœur Free5GC

La consommation de ressource de l'hôte augmente également de façon modérée lors de l'activation de TLS sur les fonctions, d'environ 14% à 30% pour le CPU et 30% pour la RAM.

La durée d'établissement des sessions PDU suit la même tendance avec une augmentation du temps nécessaire lorsque TLS est activé (Figure 24 et Tableau 2).

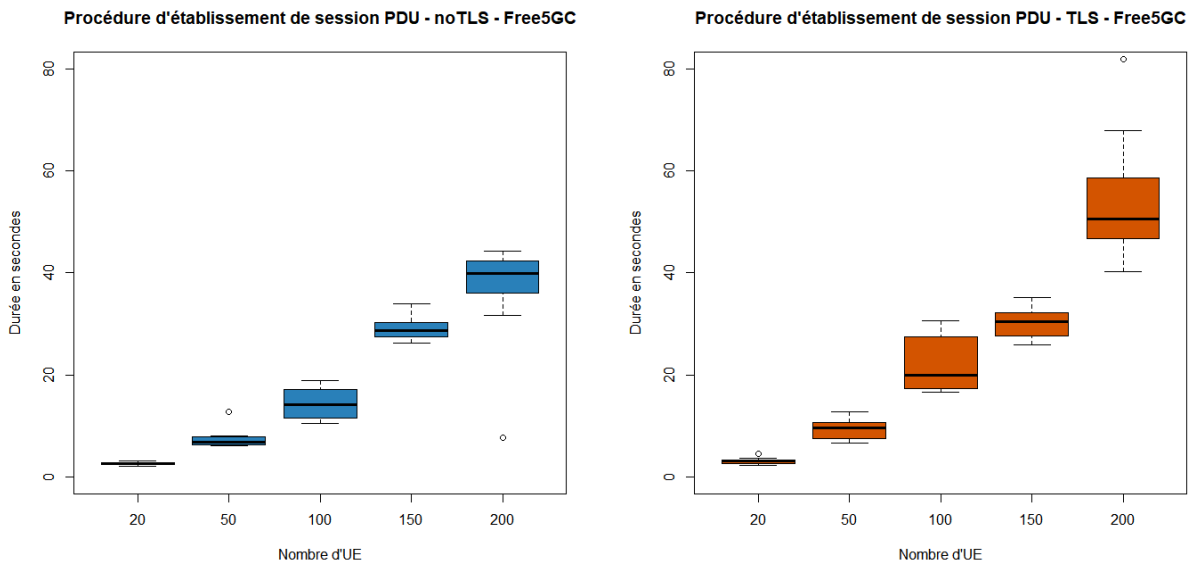


Figure 24 - Répartition des données de mesure - Procédures d'établissement de session PDU - Free5GC

	Durée moyenne* de l'établissement des sessions PDU		Delta NoTLS vs. TLS
Nombre d'UE	TLS désactivé	TLS activé	
20	2,685 secondes	3,183 secondes	+18,54 %
50	7,517 secondes	9,437 secondes	+25,54 %
100	14,505 secondes	22,292 secondes	+53,69 %
150	29,089 secondes	30,256 secondes	+4,01 %
200	36,476 secondes	53,791 secondes	+47,47 %

Tableau 2 - Analyse de la durée des procédures d'établissement de session PDU - Free5GC

En ce qui concerne Open5GS, la Figure 25 permet de constater que le temps nécessaire à la finalisation de l'enregistrement des UE reste relativement stable avec ou sans TLS. Elle montre également que la montée en charge du cœur est maitrisée, jusqu'à une certaine charge, ici 330 UE, où la durée augmente fortement. On note également que dans le cas ou TLS est actif et la charge importante, la dispersion augmente, c'est la traduction des retransmissions de messages du a une absence de réponse de l'AMF et montre un comportement inconstant du cœur.

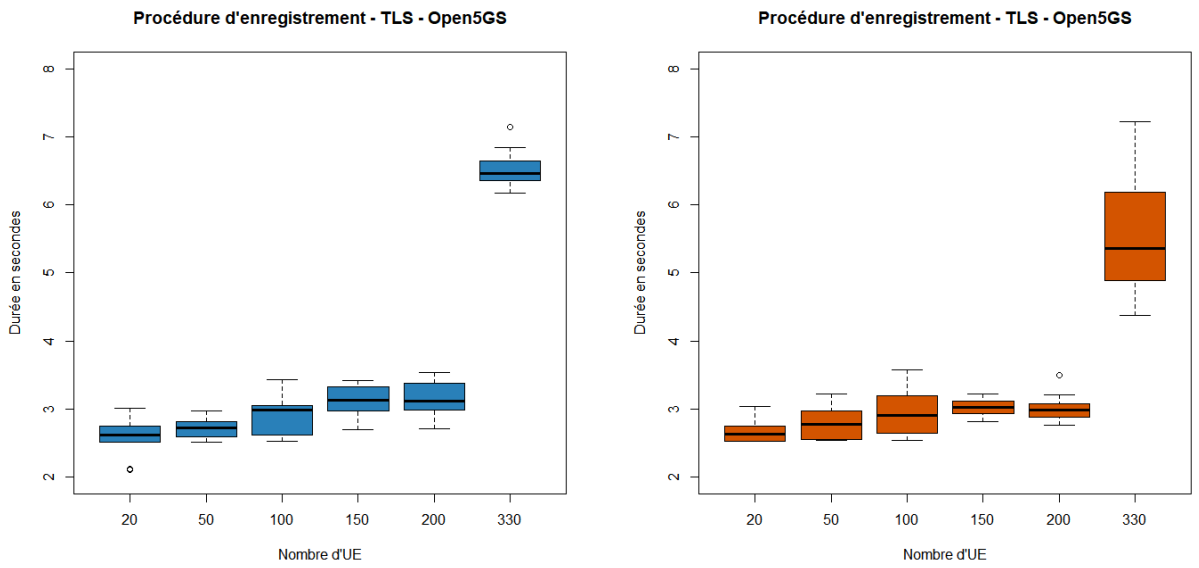


Figure 25 - Répartition des données de mesure - Procédures d'enregistrement - Open5GS

Le Tableau 3 résume ces éléments en indique l'écart entre TLS activé ou non pour les différents scénarii. Ce tableau indique également la consommation de ressources de l'hôte.

Open5GS	TLS off			TLS on			Delta NoTLS vs. TLS		
Nombre d'UEs	Durée moyenne* de l'enregistrement	Conso CPU moyenne*	Conso RAM moyenne*	Durée moyenne* de l'enregistrement	Conso CPU moyenne*	Conso RAM moyenne*	Temps de l'opération	Conso CPU moyenne	Conso RAM moyenne
20	2,585 secondes	9,47 %	2,30 GB	2,681 secondes	10,32 %	2,32GB	+3,71 %	+9,01 %	+0,7 %
50	2,731 secondes	21,04 %	2,32 GB	2,780 secondes	22,48 %	2,48 GB	+1,79 %	+6,8 %	+6,85 %
100	2,926 secondes	26,83 %	2,36 GB	2,926 secondes	29,58 %	2,50 GB	+0,00 %	+10,23 %	+10,42 %
150	3,106 secondes	41,57 %	2,39 Gb	3,016 secondes	46,59 %	2,50 GB	-2,89 %	+12,08 %	+8,61 %
200	3,143 secondes	63,18 %	2,44 GB	3,249 secondes	70,86 %	2,52 GB	+0,78 %	+12,16 %	+3,28 %
330**	6,540 secondes	51,87 %	2,50 GB	6,528 secondes	90,01 %	3,21 GB	-0,18 %	+73,53 %	+28,40 %

Tableau 3 - Analyse de la durée des procédures d'enregistrement d'UE avec et sans TLS - Cœur Open5GS

On constate que l'augmentation des besoins en RAM et CPU pour Open5GS sont modérées aux alentours de 10% ce qui montre la aussi une bonne stabilité de la plateforme. L'anomalie se présente lorsque le cœur est soumis à une forte charge ou la consommation grimpe de 73%.

Dans le cas des procédures de session PDU, le comportement du cœur est similaire comme le montre la Figure 26 et le Tableau 4, la durée est stable lorsque la charge augment jusqu'au seuil de 330 UE ou les erreurs de traitement arrivent.

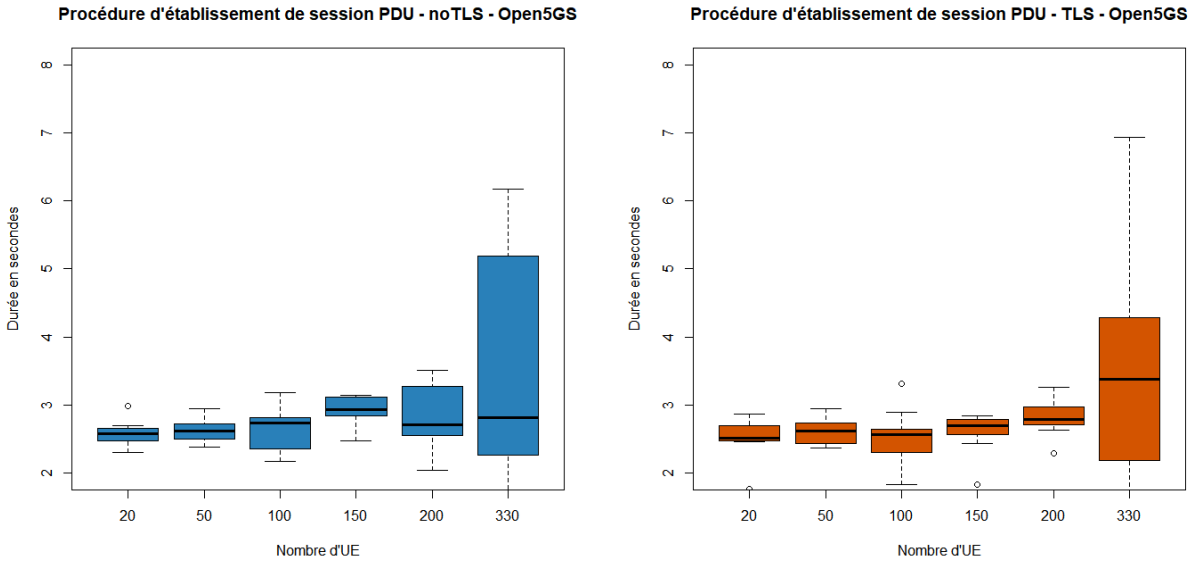


Figure 26 - Répartition des données de mesure - Procédures d'établissement de session PDU - Open5GS

	Durée moyenne de l'établissement des sessions PDU		Delta NoTLS vs. TLS
Nombre d'UE	TLS désactivé	TLS activé	
20	2,152 secondes	2,519 secondes	+17%
50	2,543 secondes	2,619 secondes	+ 2,9%
100	2,687 secondes	2,475 secondes	-7,89
150	2,928 secondes	2,610 secondes	-10%
200	2,697 secondes	2,751 secondes	+ 2%
330	3,420 secondes	5,531 secondes	+ 61,7

Tableau 4 - Analyse de la durée des procédures d'établissement de session PDU - Open5GS

A la lumière de ces résultats, on constate que le comportement des deux cœurs est relativement différent. L'utilisation des ressources par Free5GC semble sous-optimal, au prix d'une durée de procédure plus longue et qui augmente fortement lors de l'utilisation de TLS. Open5GS va au contraire utilise les ressources disponibles de façon efficace et continue de rendre le service sans souffrir de l'usage de TLS.

De plus, on constate qu'avec des ressources matérielles identiques, Open5GS présente des performances relatives supérieures à Free5GC. Cette différence peut s'expliquer par l'implémentation des deux projets, Go pour Free5GC contre C pour Open5GS. D'autre part, d'autres études (Hosseinishamoushaki, s. d.; Lando et al., 2023) mesurent cette différence de performance au profit d'Open5GS bien que l'écart soit plus réduit que dans nos résultats. Cette étude identifie également Open5GS comme un produit utilisable en production sur de petits réseaux 5G privés, alors qu'elle classe Free5GC dans les produits d'études et tests. On notera d'ailleurs que cette perspective est cohérente avec l'implémentation TLS plus aboutie d'Open5GS.

ANALYSE MICRO – IMPACT SPECIFIQUE SUR LES FONCTIONS

L'analyse micro a consisté pour nous, pour la procédure d'enregistrement d'un UE, à étudier les différences de nombre et de taille de paquets échangés entre l'AMF et l'AUSF. Les résultats sont présentés dans le Tableau 5 ci-dessous.

Conversation - noTLS							
Adresse A	**Adresse B**	**Paquets**	**Octets**	**Paquets A → B**	**Octets A → B**	**Paquets B → A**	**Octets B → A**
amf.open5gs.org	ausf.open5gs.org	14	2113	8	1091	6	1022
Conversation – TLS							
Adresse A	**Adresse B**	**Paquets**	**Octets**	**Paquets A → B**	**Octets A → B**	**Paquets B → A**	**Octets B → A**
amf.localdomain	ausf.localdomain	20	5283	11	1955	9	3331
Delta TLS vs no TLS		**43%**	**150%**	**38%**	**79%**	**50%**	**226%**

Tableau 5 - Statistiques des conversations en AMF et AUSF - Générées grâce à Wireshark

On constate que TLS à un impact significatif sur le nombre de paquets échangés entre les deux fonctions lors de la procédure d'enregistrement puisque 6 paquets supplémentaires sont transmis, soit une augmentation de l'ordre de 43%. Ces paquets sont dus à l'établissement de la session TLS réalisé par les deux fonctions en amont de leurs échanges et à l'échange des tickets de sessions comme on peut le constater dans l'extrait de capture présenté dans la Figure 27 (surlignés en violet). On retrouve les messages du « TLS Hanshake » avec le « Client hello », puis « Server Hello », contenant notamment les informations de la configuration TLS utilisée. On voit ici la négociation 1-RTT propre à TLS 1.3. Les messages utiles HTTP2 sont surlignés en vert, les clés de session ont été intégrées dans Wireshark ce qui permet d'afficher le contenu déchiffré (Voir Annexe C – Capture des échanges entre AMF et AUSF non déchiffrés).

Source	Destination	Protocol	Length	Info
amf.localdomain	ausf.localdomain	TLSv1.3	473	Client Hello (SNI=ausf.localdomain)
ausf.localdomain	amf.localdomain	TLSv1.3	1530	Server Hello, Change Cipher Spec, Er
amf.localdomain	ausf.localdomain	TLSv1.3	74	Change Cipher Spec
amf.localdomain	ausf.localdomain	TLSv1.3	142	Finished
ausf.localdomain	amf.localdomain	TLSv1.3	323	New Session Ticket
ausf.localdomain	amf.localdomain	TLSv1.3	323	New Session Ticket
ausf.localdomain	amf.localdomain	HTTP2	105	SETTINGS[0]
amf.localdomain	ausf.localdomain	HTTP2	114	Magic
amf.localdomain	ausf.localdomain	HTTP2	117	SETTINGS[0]
amf.localdomain	ausf.localdomain	HTTP2	103	WINDOW_UPDATE[0]
amf.localdomain	ausf.localdomain	HTTP2	268	HEADERS[1]: POST /nausf-auth/v1/ue-a
ausf.localdomain	amf.localdomain	HTTP2	99	SETTINGS[0]
amf.localdomain	ausf.localdomain	HTTP2/JSON	205	DATA[1], JSON (application/json)
amf.localdomain	ausf.localdomain	HTTP2	99	SETTINGS[0]
ausf.localdomain	amf.localdomain	HTTP2	210	HEADERS[1]: 201 Created
ausf.localdomain	amf.localdomain	HTTP2/JSON	377	DATA[1], JSON (application/3gpphal+j
amf.localdomain	ausf.localdomain	HTTP2	215	HEADERS[3]: PUT /nausf-auth/v1/ue-au
amf.localdomain	ausf.localdomain	HTTP2/JSON	145	DATA[3], JSON (application/json)
ausf.localdomain	amf.localdomain	HTTP2	121	HEADERS[3]: 200 OK
ausf.localdomain	amf.localdomain	HTTP2/JSON	243	DATA[3], JSON (application/json)

Figure 27 - Paquets liés à l'etablissement de session TLS entre les fonctions AMF et AUSF

Il est également intéressant de noter que l'activation de TLS sur les interfaces SBI impacte de façon importante le volume de trafic échangé qui passe de 2113 octets à 5283, soit 150% de volume supplémentaire.

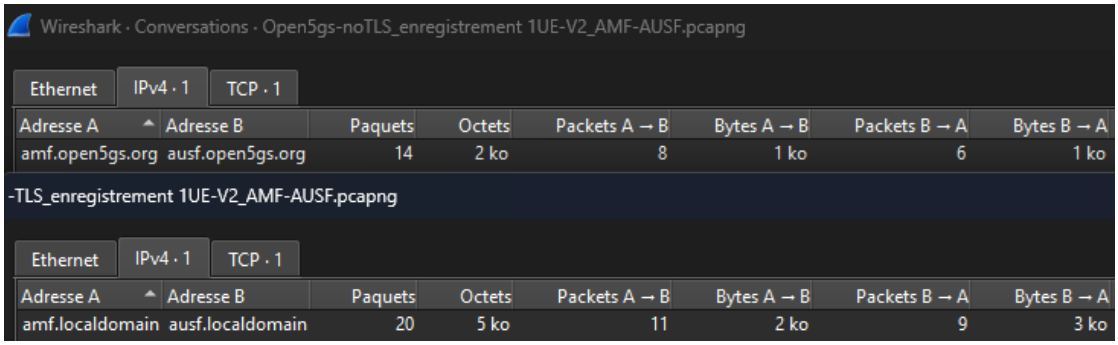


Figure 28 - Statistiques conversations Wireshark - noTLS en haut, TLS en bas

Le Tableau 6 présente la répartition des paquets par protocole toujours pour la même procédure d'attachement du mobile au réseau.

Hierarchie des protocoles - no TLS				
Protocole	**Pourcent Paquets**	**Paquets**	**Pourcent Octets**	**Octets**
Frame	100	14	100	2113
Linux cooked-mode capture	100	14	10,6	224
Internet Protocol Version 4	100	14	13,25	280
Transmission Control Protocol	100	14	21,2	448
HyperText Transfer Protocol 2	**100**	**14**	**54,95**	**1161**
Hierarchie des protocoles - TLS				
Protocole	**Pourcent Paquets**	**Paquets**	**Pourcent Octets**	**Octets**
Frame	100	20	100	5286
Linux cooked-mode capture	100	20	6,05	320
Internet Protocol Version 4	100	20	7,57	400
Transmission Control Protocol	100	20	12,11	640
Transport Layer Security	**100**	**20**	**74,27**	**3926**
HyperText Transfer Protocol 2	70	14	21,96	1161

Tableau 6 - Statistiques par type de protocole - Générées grâce à Wireshark

Ici, il est intéressant de noter le coût de TLS concernant le volume de trafic, puisqu'environ 74% du volume échangé est lié à HTTP2+TLS soit un total de 3926 octets, alors que sans TLS, le trafic HTTP2 ne représente qu'environ 55% du total, comme on peut le voir en comparant les lignes en gras. Le surcout de TLS est donc d'environ 20% en ce qui concerne le trafic utile.

Wireshark · Statistiques de la Hiérarchie des Protocoles · Open5gs-noTLS_enregistrement 1UE-V2_AMF-AUSF.pcapng

Protocole	Pourcent Paquets	Paquets	Pourcent Octets	Octets
Frame	100.0	14	100.0	2113
Linux cooked-mode capture	100.0	14	10.6	224
Internet Protocol Version 4	100.0	14	13.3	280
Transmission Control Protocol	100.0	14	21.2	448
HyperText Transfer Protocol 2	100.0	14	54.9	1161

Wireshark · Statistiques de la Hiérarchie des Protocoles · Open5gs-TLS_enregistrement 1UE-V2_AMF-AUSF.pcapng

Protocole	Pourcent Paquets	Paquets	Pourcent Octets	Octets
Frame	100.0	20	100.0	5286
Linux cooked-mode capture	100.0	20	6.1	320
Internet Protocol Version 4	100.0	20	7.6	400
Transmission Control Protocol	100.0	20	12.1	640
Transport Layer Security	100.0	20	74.3	3926
HyperText Transfer Protocol 2	70.0	14	22.0	1161

Figure 29 - Hiérarchie des protocoles - noTLS en haut, TLS en bas

6.3 Conclusion

Comme nous l'avons vu en détail dans les résultats des tests de conformités, aucun des deux projets testés ne respecte complètement les normes de 3GPP en matière d'exigences d'implémentation de TLS pour la sécurisation des communications dans le cœur. Free5GC en est le plus éloigné et le support de version de TLS obsolètes et de suites cryptographiques faible présente un risque de sécurité. Open5GS s'en sort mieux car son implémentation de TLS est plus aboutie, bien qu'on note que certaines exigences du 3GPP ne soient pas satisfaites, notamment des suites de chiffrement manquantes, mais c'est également le cas de Free5GC.

Les points d'amélioration évident concernent bien entendu la modification de l'implémentation des méthodes faisant appel aux librairies TLS externes pour les deux projets. Ce point est notamment vrai pour Free5GC et sont implémentation de crypto/tls dont les fonctions sont utilisées pour la configuration TLS des SBI des fonctions.

Free5GC utilise depuis sa version 3.4.2 du 3 juillet 2024 la version 1.21 de Go comme on peut le voir sur la Figure 30 présentant un extrait de la roadmap de Free5GC. On peut confirmer cette information sur le GitHub du projet dont la version de Go est mentionnée.

July 3, 2024: v3.4.2

The free5GC v3.4.2 includes a Go version update to 1.21 and refactoring all Network Functions. New features include setting static IPs for UEs and OAuth2 authentication for the webconsole, plus a ULCL example in free5gc-compose. Bug fixes and a new commit message check are also included.

[Refactor]

- Go version bump to Go1.21
- Refactor NFs for preparation upgrading openapi to Release17

Figure 30 - Roadmap Free5GC - Source : https://free5gc.org/history/#april-22-2025-v401

La documentation de crypto/tls pour Go 1.21 indique que faute de fournir les suites cryptographiques à utiliser, une liste de suite sera proposée par défaut, cette liste étant par ailleurs évolutive en fonction des versions de crypto/tls (Figure 31). Cela indique également que ces suites sont une liste de suites compatibles avec TLS de 1.0 à 1.2 ainsi que les suites TLS 1.3 (non configurable).

```
// CipherSuites is a list of enabled TLS 1.0–1.2 cipher suites. The order of
// the list is ignored. Note that TLS 1.3 ciphersuites are not configurable.
//
// If CipherSuites is nil, a safe default list is used. The default cipher
// suites might change over time.
CipherSuites []uint16
```

Figure 31 - Extrait de la documentation de crypto/tls a propos du choix des suites cryptographiques - Source : https://pkg.go.dev/crypto/tls@go1.21.0#pkg-functions

On apprend dans cette même documentation que par défaut, la version TLS 1.0 est la version minimale supportée par un serveur (Figure 32) ce qui n'est pas conforme avec les recommandations 3GPP.

```
// MinVersion contains the minimum TLS version that is acceptable.
//
// By default, TLS 1.2 is currently used as the minimum when acting as a
// client, and TLS 1.0 when acting as a server. TLS 1.0 is the minimum
// supported by this package, both as a client and as a server.
//
// The client-side default can temporarily be reverted to TLS 1.0 by
// including the value "x509sha1=1" in the GODEBUG environment variable.
// Note that this option will be removed in Go 1.19 (but it will still be
// possible to set this field to VersionTLS10 explicitly).
MinVersion uint16
```

Figure 32 - Extrait de la documentation de crypto/tls v1.21.0 indique que la version minimum de TLS est 1.0 pour un serveur – Source : https://pkg.go.dev/crypto/tls@go1.21.0#pkg-functions

Open5GS quant à lui nécessite seulement l'ajout de certaines suites pour être totalement conforme mais sa posture de sécurité est bonne puisque si on considère les suites proposées par les NFs, elles sont toutes référencées dans les exigences du 3GPP.

D'autre part, nous avons pu mesurer que l'utilisation de TLS dans le cœur avait un impact significatif sur les performances et/ou la consommation de ressource. De plus, le volume de trafic échangé dans le plan de contrôle augmente lors de l'utilisation de TLS pour chiffrer les communications HTTP2.

Comme nous l'avons rappelé dans le chapitre 3.4, bien que 3GPP impose aux constructeurs et éditeurs de matériel et logiciels l'implémentation de TLS dans leurs produits, son activation dans le déploiement d'un cœur de réseau n'est pas obligatoire. Les opérateurs peuvent faire le choix de sécuriser plutôt la périphérie du réseau de cœur, ce qui peut être un choix pertinent en fonction des cas d'usages et des contraintes de mise en œuvre et de production.

7 Références

3GPP. (2020). *Network Domain Security (NDS); IP network layer security* (Technical Specification (TS) No. 33.210). 3rd Generation Partnership Project (3GPP).

3GPP. (2024). *System architecture for the 5G System (5GS)* (Technical Specification (TS) No. 23.501). 3rd Generation Partnership Project (3GPP). http://www.3gpp.org/DynaReport/23501.htm

Agiwal, M., Roy, A., & Saxena, N. (2016). Next Generation 5G Wireless Networks : A Comprehensive Survey. *IEEE Communications Surveys & Tutorials*, *18*(3), 1617-1655. IEEE Communications Surveys & Tutorials. https://doi.org/10.1109/COMST.2016.2532458

Agrawal, R. (2023, février 17). A dive into TLS 1.3 Architecture and its advantages over TLS 1.2. *Medium*. https://medium.com/@akarX23/a-dive-into-tls-1-3-architecture-and-its-advantages-over-tls-1-2-2f552de24fa0

Alemu, S. B. (2020). *The transformation of TLSfrom version 1.2 to 1.3Efficiency vs Security vs Interoperability*.

Ali, A., & Singh, V. P. (2023). Comparative Analysis of Transport Layer Security (TLS) Versions. *International Journal for Research in Applied Science and Engineering Technology*, *11*(12), 680-684. https://doi.org/10.22214/ijraset.2023.57430

Allen, C., & Dierks, T. (1999, janvier). *The TLS Protocol Version 1.0* (Request for Comments, Numéro 2246). RFC Editor. https://www.rfc-editor.org/info/rfc2246

ANSSI. (2020). *Recommandations de sécurité relatives à TLS* (No. SDE-NT-35/ANSSI/SDE/NP).

Architecture SBA du réseau 5G : Microservices et SOA | Frédéric Launay. (2021, février 26). https://blogs.univ-poitiers.fr/f-launay/2021/02/26/architecture-sba-du-reseau-5g-microservices-et-soa/

Arkko, J. (2017, septembre 14). *Service-Based Architecture in 5G*. Ericsson.Com. https://www.ericsson.com/en/blog/2017/9/service-based-architecture-in-5g

ARUNRUANGSIRILERT, K., WEI, B., SONG, H., & KATTO, J. (2022). Performance Evaluation of Low-Latency Live Streaming of MPEG-DASH UHD video over Commercial 5G NSA/SA Network. *2022 International Conference on Computer Communications and Networks (ICCCN)*, 1-6. https://doi.org/10.1109/ICCCN54977.2022.9868877

Bolan, D. (2023, juillet 26). *5G Advanced—What does it mean for the 5G Core market?* Dell'Oro Group. https://www.delloro.com/5g-advanced-what-does-it-meanfor-the-5g-core-market/

Brown, G. (2017). Service-Based Architecture for 5G Core Networks. *HEAVY READING*.

Dierks, T., & Rescorla, E. (2006, avril). *The Transport Layer Security (TLS) Protocol Version 1.1* (Request for Comments, Numéro 4346). RFC Editor. https://www.rfc-editor.org/info/rfc4346

Hosseinishamoushaki, S. (s. d.). *Comparative Performance Analysis of free5GC and Open5GS Core Networks Using UERANSIM Integration*.

ITU-R. (2015). *IMT Vision – Framework and overall objectives of the future development of IMT for 2020 and beyond*.

Jost, C., & Smets, B. (2020, août 21). *Security for 5G Service-Based Architecture*. Ericsson.Com. https://www.ericsson.com/en/blog/2020/8/security-for-5g-service-

based-architecture

Lando, G., Schierholt, L. A. F., Milesi, M. P., & Wickboldt, J. A. (2023). Evaluating the performance of open source software implementations of the 5G network core. *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*, 1-7. https://doi.org/10.1109/NOMS56928.2023.10154399

Lee, H., Kim, D., & Kwon, Y. (2021). TLS 1.3 in Practice:How TLS 1.3 Contributes to the Internet. *Proceedings of the Web Conference 2021*, 70-79. https://doi.org/10.1145/3442381.3450057

Martins, J. (2023, avril 3). Using Mutual TLS To Enforce Zero Trust In The 5G Core. *Josue Martins In Cyber Security*. https://medium.com/josue-martins/using-mutual-tls-to-enforce-zero-trust-in-the-5g-core-86c99ad2372a

Moriarty, K., & Farrell, S. (2021, mars). *Deprecating TLS 1.0 and TLS 1.1* (Request for Comments, Numéro 8996). RFC Editor. https://www.rfc-editor.org/info/rfc8996

Rescorla, E. (2018, août). *The Transport Layer Security (TLS) Protocol Version 1.3* (Request for Comments, Numéro 8446). RFC Editor. https://datatracker.ietf.org/doc/html/rfc8446

Rescorla, E., & Dierks, T. (2008). *The Transport Layer Security (TLS) Protocol Version 1.2* (Request for Comments No. RFC 5246). Internet Engineering Task Force. https://doi.org/10.17487/RFC5246

Sultan, A. (2022, août 22). *5G System Overview*. https://www.3gpp.org/technologies/5g-system-overview

Tls package—Crypto/tls—Go Packages. (s. d.). Consulté 26 juin 2025, à l'adresse https://pkg.go.dev/crypto/tls#Config

Tripathi, A., & Weissberger, A. J. (2022, janvier 1). *5G Security explained : 3GPP 5G core network SBA and Security Mechanisms – IEEE ComSoc Technology Blog*. https://techblog.comsoc.org/2022/01/01/5g-network-security-threats-and-3gpp-security-mechanisms/

Why Is TLS 1.3 Better And Safer Than TLS 1.2? (2025, mars 25). *AppViewX*. https://www.appviewx.com/blogs/why-is-tls-1-3-better-and-safer-than-tls-1-2/

Xiaoyang, L. (2021, juillet 1). *What Is Network Slicing? How Does It Work?* Huawei. https://info.support.huawei.com/info-finder/encyclopedia/en/Network+Slicing.html

8 Exemple de fichier docker-compose

Le fichier compose ci-dessous est celui qui permet de lancer le cœur Free5GC avec TLS activé. Il est centré sur le bloc qui permet de provisionner le conteneur de l'AMF. Les principaux paramètres sont commentés.

On voit également que les autres NFs sont présentes (réduites) ainsi que la section permettant de configurer le réseau pour les conteneurs.

docker-compose-tls.yaml

C: > Users > alexa > AppData > Local > Temp > scp26520 > home > alex > free5gc-compose-2VM > docker-compose-tls.yaml

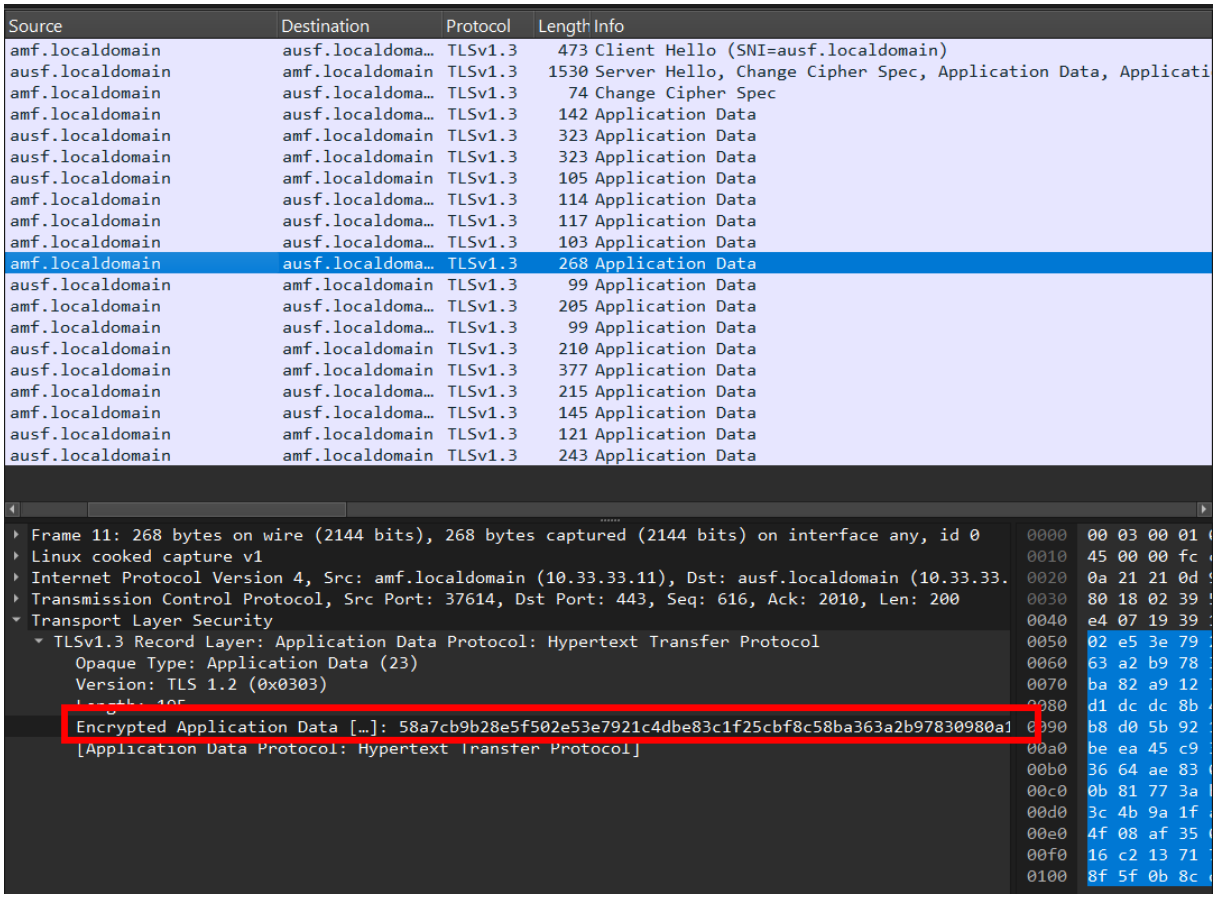
```yaml
services:

  free5gc-amf:
    container_name: amf  # <= nom que porte le conteneur dans le déploiement
    image: free5gc/amf:v4.0.1  # <= image chargée depuis le Docker Hub
    command: ./amf -c ./config/amfcfg.yaml  # <= commande de démarrage du conteneur qui charge la configuration
    expose:
      - "8000"
      - "38412:38412/sctp"
    volumes:
      - ./config-TLS/amfcfg.yaml:/free5gc/config/amfcfg.yaml   #<= on monte un volume nommé lié au fichier de config dans le conteneur
      - ./cert:/free5gc/cert            # <= Idem pour les certificats fournis
    environment:
      GIN_MODE: release
    ports:
      - "38412:38412/sctp"     #  <= le port sctp 38412 est exposé sur l'hote afin de connecter GNB
    networks:
      privnet:
        ipv4_address: 10.100.210.16   # <= on fixe l'adresse IP de la fonction pour une analyse plus facile des dumps
        aliases:
          - amf.free5gc.org   # <= la fonction est nomée afin d'être appelée facilement depuis les autres nfs
    depends_on:
      - free5gc-nrf  # <= la fonction ne demarre pas tant que la NRF n'est pas démarrée

  free5gc-ausf: ···

  free5gc-nssf: ···

  free5gc-pcf: ···

  free5gc-smf: ···

  free5gc-udm: ···

  free5gc-tngf: ···

  free5gc-nef: ···

  free5gc-webui: ···

networks:  # <= cofniguration d'un network pour les conteneurs
  privnet:
    ipam:
      driver: default
      config:
        - subnet: 10.100.210.0/24 # <= subnet affecté au network
    driver_opts:
      com.docker.network.bridge.name: br-free5gc-2vm # <= nom du network
```

9 Annexe B – Sécurité des suites cryptographiques

Le tableau ci-dessous indique le niveau de sécurité des différentes suites cryptographiques relevées sur les cœurs de réseaux 5G testés.

Secure	TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
Secure	TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
Secure	TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256
Weak	TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA
Weak	TLS_RSA_WITH_AES_128_GCM_SHA256
Weak	TLS_RSA_WITH_AES_256_GCM_SHA384
Weak	TLS_RSA_WITH_AES_256_CBC_SHA
Weak	TLS_RSA_WITH_AES_128_CBC_SHA
Weak	TLS_RSA_WITH_3DES_EDE_CBC_SHA
Weak	TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA
Weak	TLS_ECDHE_RSA_WITH_3DES_EDE_CBC_SHA
Recommended	TLS_AES_256_GCM_SHA384
Recommended	TLS_CHACHA20_POLY1305_SHA256
Recommended	TLS_AES_128_GCM_SHA256
Weak	TLS_DHE_RSA_WITH_AES_128_GCM_SHA256
Recommended	TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
Weak	TLS_DHE_PSK_WITH_AES_128_GCM_SHA256
Recommended	TLS_ECDHE_PSK_WITH_AES_128_GCM_SHA256
Recommended	TLS_ECDHE_PSK_WITH_AES_256_GCM_SHA384

Source : https://ciphersuite.info/

10 Annexe C – Capture des échanges entre AMF et AUSF non déchiffrés

La figure ci-dessous présente la même capture que celle présentée dans le chapitre résultats – Analyse micro, mais sans déchiffrement, on constate que le trafic est illisible.